

# Natural Language Processing

*II Year*

ANDREA SPINELLI

*andrea.spinelli@community.unipa.it*<sup>1</sup>

<sup>1</sup>*Computer Science Engineering Department, Università degli Studi di Palermo, Italy*

*April 20, 2026*

## Abstract

Questo documento fornisce una sintesi strutturata dei fondamenti e delle architetture avanzate del Natural Language Processing (NLP). Il testo parte dall'elaborazione testuale di base (come tokenizzazione e normalizzazione), per poi esplorare i modelli statistici a N-grammi e l'etichettatura di sequenze (POS tagging e NER) tramite algoritmi classici come HMM e CRF. Il nucleo centrale degli appunti è dedicato alla semantica vettoriale e all'evoluzione neurale: dalle reti RNN e LSTM fino alla rivoluzionaria architettura Transformer. Vengono analizzati nel dettaglio i Masked Language Models come BERT e i Large Language Models generativi (famiglia GPT, LLaMA, Mistral), includendo tecniche di prompt engineering, model alignment e metriche di valutazione. Infine, il documento affronta i sistemi di Retrieval-Augmented Generation (RAG) per limitare le allucinazioni e arricchire i modelli con dati esterni.

**Nota importante:** Si precisa che i presenti appunti costituiscono esclusivamente uno strumento di supporto allo studio e non sostituiscono in alcun modo i materiali ufficiali, le dispense e i testi di riferimento previsti dal corso universitario.

**Keywords:** *Natural Language Processing; Text Processing; N-grams; Sequence Labeling; Word Embeddings; Transformers; Large Language Models (LLM); Prompt Engineering; Model Alignment; Retrieval-Augmented Generation (RAG).*

# Contents

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                    | <b>6</b>  |
| 1.1      | Cos'è il Natural Language Processing . . . . .         | 6         |
| 1.2      | Obiettivi del corso . . . . .                          | 6         |
| 1.3      | Competenze acquisite . . . . .                         | 6         |
| 1.4      | Prospettive applicative . . . . .                      | 6         |
| <b>2</b> | <b>Text Processing</b>                                 | <b>8</b>  |
| 2.1      | Regular Expressions . . . . .                          | 8         |
| 2.2      | Words and Corpora . . . . .                            | 9         |
| 2.2.1    | Wordform and Lemma . . . . .                           | 9         |
| 2.2.2    | Corpora . . . . .                                      | 9         |
| 2.3      | Words Tokenization . . . . .                           | 10        |
| 2.3.1    | Byte Pair Encoding . . . . .                           | 10        |
| 2.3.2    | Words Normalization . . . . .                          | 12        |
| <b>3</b> | <b>N-Grams Language Model</b>                          | <b>14</b> |
| 3.1      | Evaluation and Perplexity . . . . .                    | 15        |
| 3.2      | Generalization and Zeros . . . . .                     | 16        |
| 3.3      | Smoothing: Add-one (Laplace) smoothing . . . . .       | 17        |
| 3.4      | Interpolation, Backoff, and Web-Scale LMs . . . . .    | 19        |
| 3.4.1    | Linear Interpolation . . . . .                         | 19        |
| 3.4.2    | Kneser–Ney Interpolation . . . . .                     | 20        |
| 3.4.3    | Web-scale LMs . . . . .                                | 21        |
| <b>4</b> | <b>POS &amp; NER</b>                                   | <b>22</b> |
| 4.1      | Part-of-Speech (POS) Tagging . . . . .                 | 22        |
| 4.1.1    | Closed-Class vs. Open-Class Words . . . . .            | 22        |
| 4.2      | Named Entity Recognition (NER) . . . . .               | 24        |
| 4.3      | Standard Algorithms for NER/POS Tagging . . . . .      | 25        |
| 4.3.1    | Hidden Markov Models . . . . .                         | 25        |
| 4.3.2    | Conditional Random Fields (CRF) . . . . .              | 26        |
| <b>5</b> | <b>Vectorial Semantic and Embeddings</b>               | <b>28</b> |
| 5.1      | Definitions . . . . .                                  | 28        |
| 5.1.1    | WordNet . . . . .                                      | 29        |
| 5.1.2    | FrameNet . . . . .                                     | 30        |
| 5.1.3    | Sentiment Connotation . . . . .                        | 31        |
| 5.2      | Vector Semantics . . . . .                             | 32        |
| <b>6</b> | <b>Neural Language Models</b>                          | <b>34</b> |
| 6.1      | Recurrent Neural Networks (RNN) . . . . .              | 34        |
| 6.1.1    | LSTM . . . . .                                         | 35        |
| 6.1.2    | Advanced Architectures . . . . .                       | 36        |
| 6.1.3    | Encoder-Decoder Architecture (Seq2Seq) . . . . .       | 36        |
| 6.2      | Attention . . . . .                                    | 37        |
| 6.2.1    | Funzionamento . . . . .                                | 37        |
| 6.3      | Transformers . . . . .                                 | 38        |
| 6.3.1    | Dagli Embedding Statici a quelli Contestuali . . . . . | 38        |
| 6.3.2    | Il Meccanismo di Self-Attention . . . . .              | 39        |
| 6.3.3    | Multi-Head Attention . . . . .                         | 39        |
| 6.3.4    | Mascheramento (Masking) . . . . .                      | 39        |
| 6.3.5    | Il Blocco Transformer . . . . .                        | 40        |

|           |                                                             |           |
|-----------|-------------------------------------------------------------|-----------|
| 6.3.6     | Input: Token e Position Embeddings . . . . .                | 40        |
| 6.3.7     | Output: La Language Modeling Head . . . . .                 | 40        |
| <b>7</b>  | <b>Masked Language Models and Fine Tuning</b>               | <b>42</b> |
| 7.1       | BERT . . . . .                                              | 42        |
| 7.1.1     | Pre-training di BERT . . . . .                              | 43        |
| 7.1.2     | Embedding Contestuali . . . . .                             | 44        |
| 7.2       | Fine-Tuning . . . . .                                       | 45        |
| 7.2.1     | Classificazione di Sequenze . . . . .                       | 45        |
| 7.2.2     | Sequence Labeling (es. NER) . . . . .                       | 45        |
| 7.3       | Varianti di BERT . . . . .                                  | 45        |
| <b>8</b>  | <b>Large Language Models</b>                                | <b>46</b> |
| 8.1       | Architetture . . . . .                                      | 46        |
| 8.2       | Decoder-only Models e Conditional Generation . . . . .      | 46        |
| 8.3       | Decoding e Strategie di Campionamento . . . . .             | 47        |
| 8.3.1     | Decoding Deterministico . . . . .                           | 47        |
| 8.3.2     | Decoding Stocastico (Sampling) . . . . .                    | 48        |
| 8.4       | Pre-training . . . . .                                      | 49        |
| 8.4.1     | Dati di Pre-training . . . . .                              | 49        |
| 8.5       | Fine-Tuning dei LLM . . . . .                               | 49        |
| 8.5.1     | Parameter-Efficient Fine Tuning (PEFT) . . . . .            | 49        |
| 8.6       | Valutazione dei LLM . . . . .                               | 49        |
| 8.6.1     | Log-Likelihood e Perplexity . . . . .                       | 49        |
| 8.7       | Scalabilità e Tecniche di Ottimizzazione . . . . .          | 50        |
| 8.8       | Leggi di Scaling (Scaling Laws) . . . . .                   | 50        |
| 8.8.1     | KV Cache . . . . .                                          | 50        |
| 8.8.2     | LoRA (Low-Rank Adaptation) . . . . .                        | 50        |
| 8.8.3     | Quantizzazione . . . . .                                    | 50        |
| <b>9</b>  | <b>Common LLM Architectures</b>                             | <b>52</b> |
| 9.1       | Architettura Mixture of Experts (MoE) . . . . .             | 52        |
| 9.2       | La Famiglia GPT . . . . .                                   | 54        |
| 9.2.1     | Pre-training e Fine-tuning in GPT . . . . .                 | 54        |
| 9.3       | Architetture Ottimizzate: LLaMA e Mistral . . . . .         | 55        |
| 9.3.1     | LLaMA (Meta) . . . . .                                      | 55        |
| 9.3.2     | Mistral . . . . .                                           | 55        |
| 9.4       | Altri Modelli e Approcci . . . . .                          | 55        |
| <b>10</b> | <b>Prompting</b>                                            | <b>56</b> |
| 10.1      | In-Context Learning e Induction Head . . . . .              | 56        |
| 10.2      | Model Alignment . . . . .                                   | 56        |
| 10.2.1    | Instruction Tuning (SFT) . . . . .                          | 56        |
| 10.3      | Tecniche Avanzate di Prompting . . . . .                    | 57        |
| 10.3.1    | Chain-of-Thought (CoT) . . . . .                            | 57        |
| 10.3.2    | Automatic Prompt Optimization (APO) . . . . .               | 57        |
| 10.4      | Preference Alignment: RLHF e DPO . . . . .                  | 57        |
| 10.4.1    | Reinforcement Learning with Human Feedback (RLHF) . . . . . | 57        |
| 10.4.2    | Direct Preference Optimization (DPO) . . . . .              | 57        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>11 Generative Tasks &amp; Evaluation</b>               | <b>58</b> |
| 11.1 Task Generativi . . . . .                            | 58        |
| 11.1.1 Machine Translation (MT) . . . . .                 | 58        |
| 11.1.2 Text Classification e Question Answering . . . . . | 58        |
| 11.2 Valutazione dei Modelli Generativi . . . . .         | 59        |
| 11.2.1 Metriche di Valutazione Automatica . . . . .       | 59        |
| 11.2.2 LLM as a Judge . . . . .                           | 59        |
| 11.2.3 RAGAS . . . . .                                    | 60        |
| <b>12 Retrieval-Augmented Generation</b>                  | <b>62</b> |
| 12.1 Componenti dell'Architettura RAG . . . . .           | 62        |
| 12.1.1 Corpus ed Embeddings . . . . .                     | 62        |
| 12.1.2 Retriever e Vector Store . . . . .                 | 62        |
| 12.1.3 Generator . . . . .                                | 62        |
| 12.2 Valutazione dei Sistemi RAG . . . . .                | 63        |
| 12.3 Integrazioni Avanzate: Grafi e Agentic RAG . . . . . | 63        |
| 12.3.1 RAG e Knowledge Graphs . . . . .                   | 63        |
| 12.3.2 Agentic RAG . . . . .                              | 63        |



# 1 Introduction

## 1.1 Cos'è il Natural Language Processing

Il **Natural Language Processing** (NLP), o Elaborazione del Linguaggio Naturale, è l'area dell'Intelligenza Artificiale che si occupa dello studio e dello sviluppo di metodi e sistemi in grado di analizzare, comprendere e generare linguaggio umano. A differenza dei linguaggi formali o di programmazione, il linguaggio naturale è intrinsecamente ambiguo, ricco di sfumature semantiche e fortemente legato al contesto. L'NLP rappresenta dunque una disciplina di frontiera, che integra competenze linguistiche, informatiche e statistiche per trasformare testi e conversazioni in conoscenza computabile.

## 1.2 Obiettivi del corso

L'obiettivo del corso è fornire una panoramica aggiornata delle più recenti tecnologie di Intelligenza Artificiale applicate al linguaggio. In particolare, si intende mostrare come un robot o un agente virtuale possano acquisire la capacità di comprendere ed elaborare il linguaggio umano per diversi scopi applicativi.

## 1.3 Competenze acquisite

Alla fine del corso, lo studente sarà in grado di comprendere e utilizzare strumenti che permettono di:

- **Rispondere a domande** tramite modelli di *question answering*, capaci di cercare e sintetizzare informazioni in modo coerente.
- **Riconoscere la struttura di un testo**, ad esempio individuando frasi, dipendenze sintattiche e categorie grammaticali.
- **Individuare tratti emotivi e di genere** all'interno di un documento, sfruttando tecniche di *sentiment analysis* e classificazione.
- **Tradurre tra lingue diverse**, mediante modelli neurali di traduzione automatica basati su architetture Transformer.
- **Interagire in modo fluente con gli utenti**, integrando modelli di dialogo e assistenti conversazionali.

## 1.4 Prospettive applicative

Queste competenze trovano applicazione in numerosi contesti: dai sistemi di assistenza virtuale all'analisi automatica di grandi collezioni di testi, fino alla creazione di strumenti multilingue per la comunicazione e la ricerca. Il corso costituisce dunque una base solida per chi desidera approfondire il ruolo delle tecnologie linguistiche nella società digitale contemporanea.



## 2 Text Processing

Questa sezione introduce gli strumenti di base per preparare un corpus testuale: espressioni regolari, tokenizzazione (parole e frasi), normalizzazione e subword tokenization (BPE/WordPiece). Useremo la notazione per sequenze  $w_{1:n}$  e indicheremo con  $N$  il numero di token e con  $|V|$  la dimensione del vocabolario.

### 2.1 Regular Expressions

Le espressioni regolari (*regex*) forniscono un linguaggio compatto per descrivere insiemi di stringhe. Sono essenziali per filtrare, validare e trasformare testo prima di applicare modelli statistici o neurali. Un uso tipico è intercettare varianti ortografiche e maiuscole, ad esempio `[Ww]oodchuck` cattura sia “woodchuck” sia “Woodchuck”.

Gli operatori chiave includono classi di caratteri, disgiunzioni, quantificatori, ancoraggi e gruppi; le asserzioni di *lookaround* vincolano il contesto senza consumare caratteri. Nella pratica bilanciamo *precision* e *recall*: pattern troppo larghi producono *falsi positivi*, pattern troppo stretti producono *falsi negativi*.

| Funzione            | Sintassi                         | Descrizione                        | Esempio (match)                                                  |
|---------------------|----------------------------------|------------------------------------|------------------------------------------------------------------|
| Classe di caratteri | <code>[A-Z]</code>               | Una lettera maiuscola              | <code>Regex</code> → <code>R</code>                              |
| Classe negata       | <code>[^Ss]</code>               | Qualsiasi tranne “s/S”             | <code>bus</code> → <code>bu</code> ( <i>dipende dal motore</i> ) |
| Disgiunzione        | <code>cat dog</code>             | “cat” oppure “dog”                 | <code>hotdog</code> → <code>dog</code>                           |
| Quantificatore opz. | <code>x?</code>                  | 0 o 1 occorrenza di <code>x</code> | <code>colou?r</code> → <code>colour</code> , <code>color</code>  |
| Quantificatore *    | <code>x*</code>                  | 0 o più occorrenze                 | <code>ba*</code> → <code>b</code> , <code>baaa</code>            |
| Quantificatore +    | <code>x+</code>                  | 1 o più occorrenze                 | <code>ba+</code> → <code>baa</code>                              |
| Intervallo          | <code>x{m,n}</code>              | Da <i>m</i> a <i>n</i> occorrenze  | <code>a{2,3}</code> → <code>aa</code> , <code>aaa</code>         |
| Ancoraggi           | <code>^</code> , <code>\$</code> | Inizio/fine riga                   | <code>^The</code> su “The ...”                                   |

Table 1: Matching

ELIZA (Weizenbaum, 1966) dimostrò come semplici regole di pattern-matching e sostituzione possano simulare una conversazione nello stile rogersiano: pattern come `.* I AM (depressed|sad) .*` venivano riscritti in risposte riflessive (“WHY DO YOU THINK YOU ARE \1”). È un caso emblematico del ruolo delle regex/riscritture come baseline conversazionale.

| Funzione              | Sintassi                          | Descrizione                     | Esempio (match)                                          |
|-----------------------|-----------------------------------|---------------------------------|----------------------------------------------------------|
| Confine di parola     | <code>\b</code>                   | Bordo parola                    | <code>\bdog\b</code> → “dog” isolato                     |
| Gruppo catturante     | <code>(...)</code>                | Raggruppa e memorizza           | <code>(\d+)-(\d+)</code> su <code>12-34</code>           |
| Backreference         | <code>\1</code> , <code>\2</code> | Richiama gruppi precedenti      | <code>(\w+) \1</code> → “go go”                          |
| Gruppo non catturante | <code>(?:...)</code>              | Raggruppa senza memorizzare     | <code>(?:pre re)fix</code>                               |
| Lookahead positivo    | <code>(?=...)</code>              | Richiede contesto seguente      | <code>\w+(?=\.)</code> su “word.” → <code>word</code>    |
| Lookahead negativo    | <code>(?!...)</code>              | Esclude contesto seguente       | <code>Dog(?!s)</code> non matcha “Dogs”                  |
| Lookbehind positivo   | <code>(?&lt;=...)</code>          | Richiede contesto precedente    | <code>(?&lt;=EU)\d+</code> su “EU123” → <code>123</code> |
| Lookbehind negativo   | <code>(?&lt;!\...)</code>         | Esclude contesto precedente     | <code>(?&lt;!\Mr\.)\b[A-Z]</code> evita abbr.            |
| Sostituzione          | <code>s/pat/repl/</code>          | Rimpiazza con <code>repl</code> | <code>s/(\w+)s/\1/</code> : “dogs” → “dog”               |

Table 2: Sostituzioni, gruppi e lookahead

**Nota:** il supporto e le regole dei lookbehind variano tra i motori; alcuni richiedono ampiezza fissa.



## 2.2 Words and Corpora

Nel testo distinguiamo token e type. Un **token** è un'istanza nel flusso testuale (ogni parola “che appare”), mentre un **type** è la forma distinta nel vocabolario. Se indichiamo con  $N$  il numero di token osservati e con  $|V|$  la dimensione del vocabolario, una stessa sequenza può essere ambigua: nella frase “they lay back on the San Francisco grass and looked at the stars and their” puoi contare  $N = 14$  o  $15$  token a seconda di come tratti “San Francisco”, e i type possono variare se normalizzi maiuscole o multiword expressions (MWE).

### 2.2.1 Wordform and Lemma

Accanto a token e type servono due nozioni lessicali: **wordform** (la forma flessa in superficie, es. “cats”) e **lemma** (la radice lessicale, es. “cat”). “cat” e “cats” hanno lo stesso lemma ma wordform diverse; l'analisi a lemma è cruciale quando vuoi aggregare varianti flessive senza perdere il “significato-base”.

Empiricamente, la **Legge di Heaps** (o di **Herdan**) modella la crescita del vocabolario all'aumentare del corpus:

$$|V(N)| \approx K N^\beta, \quad 0 < \beta < 1.$$

In molti testi generali,  $\beta$  è sublineare e spesso nell'intorno 0.67–0.75: più leggi, più compaiono parole nuove, ma con un ritmo decrescente. Alcuni esempi di grandezze sono:

|                                 | Tokens = $N$ | Types = $ V $ |
|---------------------------------|--------------|---------------|
| Switchboard phone conversations | 2.4 million  | 20 thousand   |
| Shakespeare                     | 884,000      | 31 thousand   |
| COCA                            | 440 million  | 2 million     |
| Google N-grams                  | 1 trillion   | 13+ million   |

Table 3: Sample Data Sets with Tokens and Types

Un fatto correlato è la **Legge di Zipf**, secondo cui la frequenza  $f(r)$  della parola di rango  $r$  (in una lista ordinata per frequenza) decresce approssimativamente come  $f(r) \propto 1/r^\alpha$ , spesso con  $\alpha \approx 1$ . Questo spiega perché poche parole funzionali dominano i conteggi e tante parole rare appaiono pochissimo.

### 2.2.2 Corpora

Un **corpus** non è “testo neutro”: è *prodotto da autori specifici, in un certo tempo, varietà e lingua*, per una funzione comunicativa precisa. Per questo i corpora differiscono per lingua (incluse lingue con spazi vs. senza spazi), varietà (es. AAE), code-switching (esempi Spagnolo/English; Hindi/English), genere testuale (news, fiction, Wikipedia...), e demografie degli autori.

In pratica si parla di corpora bilanciati (mirano a coprire più generi in modo controllato), di monitoraggio (crescono nel tempo, es. web/news), di dominio (settoriali: clinico, legale, social media) o di riferimento (usati come standard per compiti/valutazioni). Le scelte di campionamento e filtraggio impattano le statistiche di frequenza e la trasferibilità dei modelli.

Oggi è buona prassi accompagnare un corpus con una datasheet che documenti motivazione, composizione, processo di raccolta, pre-processing, raccomandazioni d'uso e aspetti etici; questo migliora trasparenza, riproducibilità e governance del rischio.

Quando servono etichette (POS, NER, sentiment...), l'annotazione va pianificata: linee guida chiare, più annotatori, misura dell'accordo e gestione dei bias culturali/linguistici.

## 2.3 Words Tokenization

Dato un testo grezzo  $x_{1:T}$  (una sequenza di caratteri Unicode), la tokenizzazione in parole è la funzione che produce una sequenza di token  $w_{1:n}$  e una partizione per indici di confine  $1 = b_1 < \dots < b_{n+1} = T + 1$  tale che ogni token è  $w_i = x_{b_i:(b_{i+1}-1)}$ . In pratica stiamo decidendo dove spezzare (i soli “spazi” non bastano!). Questa decisione non è universale: dipende dalla lingua, dal dominio e dal task (IR, NER, traduzione, QA).

Alcuni esempi sono:

- **Punteggiatura e simboli:** I segni possono essere separati o aggregati al token: “ciao!”  $\rightarrow$  “ciao” + “!” oppure “ciao!”. La scelta va stabilita a priori e tenuta coerente, specie per POS/NER.
- **Numeri e separatori locali:** In italiano il decimale usa la virgola: “3,14”. Il punto può essere separatore delle migliaia: “1.234,56”. In inglese è spesso l’opposto.
- **Contrazioni, elisioni e clitici:** In inglese “we’re” può diventare “we” + “re” (utile per POS). In italiano hai elisioni e clitici attaccati: “l’ho”, “dell’acqua”, “portarglielo”.
- **Multiword expressions (MWE):** “San Francisco”, “New York”, “a lungo termine”, “stati uniti d’america”: per molti task è utile trattarle come un unico token logico (es. San\_Francisco).
- **URL, hashtag, @mention, email, emoji:** Spesso conviene tokenizzarli come unità atomiche (un singolo token) per non distruggere struttura e semantica.
- **Lingue senza spazi o con script complessi:** In cinese, giapponese o thai non ci sono spazi tra parole; la tokenizzazione diventa un problema di segmentazione a sé, spesso trattato come etichettatura di confine (BIO) con CRF o modelli neurali. In alternativa si lavora a caratteri o subword.

In un pipeline moderno conviene sempre distinguere: (i) **normalizzazione di base** (Unicode NFC/NFKC, spazi multipli, rimozione di caratteri di controllo), (ii) **tokenizzazione in parole**, (iii) **eventualmente subword** (BPE/WordPiece)

### 2.3.1 Byte Pair Encoding

Per modelli neurali e vocabolari aperti, la tokenizzazione a subword riduce gli OOV e cattura morfemi ricorrenti con un vocabolario finito. L’idea di **Byte Pair Encoding (BPE)** è imparare un inventario di “pezzi” di parola (subword) a partire da un corpus, fondendo iterativamente le coppie di simboli adiacenti più frequenti. All’inizio ogni parola è spezzata in caratteri più un marcatore di fine-parola (ad es. “\_” o  $\langle/w\rangle$ ). Fondendo le coppie più frequenti, si formano unità utili come radici e suffissi, riducendo gli OOV senza dover usare un vocabolario di parola enorme.

Supposto di avere un lessico di parole con frequenze  $\{(w^{(i)}, c^{(i)})\}_{i=1}^M$ . Ogni parola è rappresentata come sequenza di simboli iniziali a livello carattere, p.es. “casa”  $\mapsto$  [c, a, s, a, \_]. Definiamo  $V_0$  l’alfabeto iniziale (caratteri + marcatore di fine-parola). Ad ogni passo  $t$ :

1. Contiamo, pesati per frequenza, le coppie adiacenti di simboli in tutto il lessico.
2. Scegliamo la coppia più frequente  $(x, y)$  ed eseguiamo la fusione in un nuovo simbolo  $z = xy$ .
3. Sostituiamo tutte le occorrenze di  $x, y$  adiacenti con  $z$  e aggiorniamo l’inventario:  $V_{t+1} = V_t \cup \{z\}$ .
4. Ripetiamo finché raggiungiamo la taglia di vocabolario desiderata  $|V_T|$  (o un numero di fusioni  $T$ ). Il risultato dell’addestramento sono l’inventario  $V_T$  e soprattutto l’ordine delle fusioni  $\mathcal{M} = [(x_1, y_1), \dots, (x_T, y_T)]$ , che definisce come segmentare in test.

Dato un token di input  $w$ , lo si spezza in caratteri più marcatore e si applicano le fusioni nell’ordine imparato. Ogni volta che una coppia presente in  $\mathcal{M}$  è adiacente nella sequenza corrente, la si fonde; si procede dall’inizio alla fine della lista di fusioni, fino a quando non si possono fare altre fusioni. Questo processo è deterministico dato l’ordine dei merge.

In pratica, dopo molte fusioni, parole molto frequenti rimangono intere come singoli token; parole meno frequenti diventano concatenazioni di subword comuni (radice + suffissi).

**Ex.**

Immaginiamo un lessico con frequenze: “casa”, “case”, “casetta”, “casette”. Dopo poche fusioni probabili:

“c”+“a” → “ca”, “s”+“a” → “sa”, “t”+“t” → “tt”, “e”+“ ” → “\*\*e\*\*”

Emergeranno subword come “cas”, “etta”, “ette”, e token completi come “casa\_” se molto frequente. Così “casetta” può segmentarsi in “cas + etta + \_”, mentre “case” diventa “cas + e\_”.

Per l’italiano è comune che BPE catturi morfemi produttivi come “-zione”, “-mente”, “-ismo”, “-are/-ere/-ire”, “-iamo”, “-ato/-ito”, migliorando la generalizzazione.

Il **marcatore di fine-parola** (ad es. “” o  $\langle/w\rangle$ ) impedisce fusioni che attraversano i confini di parola e consente di distinguere “est” come parola (con “\*\*est”) dal suffisso “-est\*\*” dentro “new + est”. Senza questo segnale, le fusioni rischiano di unire pezzi indesiderati attraversando spazi.

**Snippet: BPE**

```

1  from collections import Counter
2
3  # Corpus di esempio: parole separate da spazi e simbolo _ per il fine-parola
4  corpus = ["low _", "lower _", "newest _", "widest _"]
5  # Rappresentiamo ogni parola come lista di caratteri (o subword iniziali)
6  words = [list(w.replace(" ", "")) for w in corpus]
7  # Esempio: "lowest _" -> ["l","o","w","e","s","t","_"]
8
9  def get_stats(words):
10     pairs = Counter()
11     for w in words:
12         for i in range(len(w)-1):
13             pairs[(w[i], w[i+1])] += 1
14     return pairs
15
16  def merge_pair(words, pair):
17     a, b = pair
18     merged = []
19     for w in words:
20         i = 0
21         new_w = []
22         while i < len(w):
23             if i < len(w)-1 and w[i] == a and w[i+1] == b:
24                 new_w.append(a+b) # fusione
25                 i += 2
26             else:
27                 new_w.append(w[i])
28                 i += 1
29         merged.append(new_w)
30     return merged
31
32  # Applichiamo poche fusioni
33  for _ in range(10):
34     stats = get_stats(words)
35     if not stats:
36         break
37     best = max(stats, key=stats.get)
38     words = merge_pair(words, best)
39
40  print("Vocabolario subword appreso:", sorted({s for w in words for s in w}))

```

### 2.3.2 Words Normalization

La normalizzazione porta le wordform a una rappresentazione standard prima dell'analisi. Si pensi ad esempio una trasformazione deterministica applicata ai token o all'intero testo: data una stringa Unicode  $w$ , otteniamo la forma normalizzata  $\mathcal{N}(w)$ . Due forme sono considerate equivalenti se condividono la stessa normale, cioè  $w \sim w'$  se e solo se  $\mathcal{N}(w) = \mathcal{N}(w')$ . In generale, il vocabolario collassa: se  $V$  è l'insieme dei type e  $V_{\mathcal{N}} = \{\mathcal{N}(w) : w \in V\}$ , allora  $|V_{\mathcal{N}}| \leq |V|$ .

Scriviamo la normalizzazione come composizione di passi più semplici:  $\mathcal{N} = \mathcal{N}_k \circ \dots \circ \mathcal{N}_1$ . Conviene che sia *deterministica* e *idempotente*, ossia  $\mathcal{N}(\mathcal{N}(w)) = \mathcal{N}(w)$ , in modo che il risultato non dipenda da quante volte applichiamo la procedura. Se  $c(w)$  è la frequenza della wordform  $w$  e  $c_{\mathcal{N}}(u)$  la frequenza della forma normalizzata  $u$ , allora  $c_{\mathcal{N}}(u) = \sum_{w: \mathcal{N}(w)=u} c(w)$ .

La normalizzazione include:

- **Case Folding:** Una trasformazione che riduce le varianti di maiuscole/minuscole a una forma canonica, così da unificare conteggi e diminuire la sparsità. Formalmente è una funzione  $\mathcal{C} : \Sigma^* \rightarrow \Sigma^*$  tale che, per ogni token  $w$ , la forma ridotta è  $\mathcal{C}(w)$ . La versione più comune è il lowercasing completo, che scriviamo come

$$C_{lower}(w) = lowercase(w)$$

- **Lemmatization:** Mappa una wordform alla sua voce di dizionario (il lemma) usando conoscenza morfologica e, di norma, l'informazione sul part-of-speech (POS) e sulle feature flessive (ad es. “am”, “are”, “is”  $\rightarrow$  “be”). Possiamo vederla come una funzione condizionata:

$$\mathcal{L} : (w, pos, morph) \mapsto \ell$$

- **Stemming:** Applica regole di riscrittura per troncare suffissi e talvolta prefissi senza garantire un lemma valido. Si può formalizzare come una funzione non contestuale

$$\mathcal{S} : w \mapsto s$$

definita da un insieme ordinato di regole  $r_1, \dots, r_k$  che riscrivono stringhe di coda (o testa). A differenza di  $\mathcal{L}$ ,  $\mathcal{S}$  non usa POS né morfologia; per questo è più veloce e robusto su testi rumorosi, ma può introdurre due tipi di errore: overstemming (parole semanticamente distinte convergono allo stesso stem) e understemming (varianti morfologiche restano separate).

- **Sentence Segmentation:** Individua i confini tra enunciati in un testo continuo. Partiamo da una sequenza di caratteri  $x_{1:T}$  e vogliamo stimare una sequenza di indici  $1 = s_1 < s_2 < \dots < s_{m+1} = T + 1$  tali che la  $j$ -esima frase sia  $x_{s_j:(s_{j+1}-1)}$ . Gli stessi simboli che usiamo come indizi di fine frase (“.”, “?”, “!”) sono ambigui perché compaiono nelle abbreviazioni, nei numeri e nelle sigle. Questo rende il problema un caso tipico di disambiguazione di confine.
- **Stopwords Removal:** Parole molto frequenti e poco discriminative in certi compiti (articoli, preposizioni, congiunzioni, ausiliari). L'idea classica è filtrarle prima dell'analisi per ridurre rumore e dimensionalità. Possiamo vederlo come un operatore di filtro che, dato l'insieme dei token del documento  $d$ , rimuove quelli presenti in una lista  $S$ : se  $w \in S$ , allora  $w$  non compare nella rappresentazione finale.



### 3 N-Grams Language Model

Un **language model** assegna probabilità a una sequenza di parole  $W = w_{1:N}$  e consente di prevedere la prossima parola  $P(w_i | w_{1:i-1})$ . Usando la regola della catena, la probabilità congiunta si scompone come

$$P(w_{1:N}) = \prod_{i=1}^N P(w_i | w_{1:i-1}).$$

Poiché stimare  $P(w_i | w_{1:i-1})$  con tutto il contesto è impraticabile, adottiamo l'assunzione di Markov: ogni parola dipende solo dalle ultime  $k$  parole. Un modello a  $n$ -grammi con  $n = k + 1$  approssima

$$P(w_{1:N}) \approx \prod_{i=1}^N P(w_i | w_{i-k:i-1}),$$

con i casi notevoli di unigram ( $k = 0$ ), bigram ( $k = 1$ ) e trigram ( $k = 2$ ). Per i bigrammi, la stima di **massima verosimiglianza (MLE)** è

$$\hat{P}_{\text{MLE}}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})},$$

mentre per i trigrammi è

$$\hat{P}(w_i | w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

In pratica si includono **token di inizio/fine** frase  $\langle s \rangle, \langle /s \rangle$  e si lavora in log-spazio:

$$\log P(w_{1:N}) = \sum_i \log P(\cdot)$$

Possiamo estendere il modello ai trigrammi, ai 4-grammi, ai 5-grammi.

In generale, i modelli di linguaggio bigrammi sono insufficienti, perché **il linguaggio ha dipendenze a lunga distanza**: *"Il computer che avevo appena messo nella sala macchine al quinto piano si è bloccato"*. Ma spesso possiamo cavarcela con modelli a N-grammi.

**Ex.**

Supponiamo un corpus in cui abbiamo i conteggi:  $c(\langle s \rangle, \{Io\}) = 10$ ,  $c(\{Io, studio\}) = 8$ ,  $c(\{studio, NLP\}) = 5$ ,  $c(\{NLP, \langle /s \rangle\}) = 5$ , e i conteggi di contesto  $c(\langle s \rangle) = 100$ ,  $c(\{Io\}) = 50$ ,  $c(\{studio\}) = 20$ ,  $c(\{NLP\}) = 5$ . La probabilità della frase  $\langle s \rangle$  Io studio NLP  $\langle /s \rangle$  secondo il modello bigram è

$$P \approx \hat{P}(Io | \langle s \rangle) \cdot \hat{P}(studio | Io) \cdot \hat{P}(NLP | studio) \cdot \hat{P}(\langle /s \rangle | NLP) = \frac{10}{100} \cdot \frac{8}{50} \cdot \frac{5}{20} \cdot \frac{5}{5}.$$

Basta uno zero in uno dei fattori per annullare l'intera probabilità: questo motiva lo *smoothing* e la gestione esplicita degli OOV con  $\langle UNK \rangle$ .

### 3.1 Evaluation and Perplexity

La **valutazione** di un language model a  $n$ -grammi si basa sulla probabilità assegnata a un corpus di test tenuto fuori dall'addestramento. La quantità centrale è la *log-likelihood* media per token, ovvero la *cross-entropy* empirica, da cui discende la *perplexity*.

Dato un test  $W = w_{1:N}$  e un modello  $P$ , definiamo la **log-verosimiglianza media** per token come

$$\bar{\ell}(W; P) = \frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{1:i-1}).$$

La **cross-entropy** (in nat/token) è  $H(W; P) = -\bar{\ell}(W; P)$  e misura la sorpresa media del modello; in bit/token vale  $H_2(W; P) = H(W; P) / \log 2$ . La **perplexity** è l'esponenziale della cross-entropy:

$$PP(W; P) = \exp(H(W; P)) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{1:i-1})\right).$$

Valori più bassi di  $H$  e di  $PP$  indicano un modello migliore, a parità di test e pre-processing.

La probabilità di una frase  $s$  è  $P(s) = \prod_j P(w_j | w_{1:j-1})$ ; per confrontare frasi di lunghezze diverse si usa sempre la normalizzazione per token. Su un test di  $M$  frasi con lunghezze  $N_1, \dots, N_M$  si riporta una sola metrica a livello corpus sommando i log e dividendo per  $N = \sum_m N_m$ .

**Minimizzare la perplessità equivale a massimizzare la probabilità.**

La perplexity dipende da: (i) token speciali di inizio/fine frase  $\langle s \rangle, \langle /s \rangle$  (coerenza tra training e test), (ii) gestione degli OOV con  $\langle UNK \rangle$  (*closed vocabulary*: tutte le parole non viste vanno mappate a  $\langle UNK \rangle$  anche nel test), (iii) smoothing: senza smoothing, un singolo fattore nullo rende  $PP = +\infty$ . Si lavora in log-spazio per stabilità:  $\log P(w_{1:N}) = \sum_i \log P(\cdot)$ .

**Ex.**

Su una frase di  $N = 4$  token, con probabilità condizionate 0.20, 0.40, 0.50, 0.25, la log-likelihood media è  $\bar{\ell} = \frac{1}{4}(\log 0.20 + \log 0.40 + \log 0.50 + \log 0.25)$ , la cross-entropy  $H = -\bar{\ell}$  e la perplexity

$$PP = \exp(-\bar{\ell}) = \exp\left(-\frac{1}{4}(\log 0.20 + \log 0.40 + \log 0.50 + \log 0.25)\right) \approx 2.51.$$

In base 2 si riporta  $H_2 = H / \log 2$  (bit/token) e  $PP_2 = 2^{H_2}$ .

**Confronto tra modelli e significatività** Con due LM  $P$  e  $Q$  addestrati sullo stesso training, la differenza di cross-entropy sul test è

$$\Delta H = H(W; P) - H(W; Q) = -\frac{1}{N} \sum_{i=1}^N (\log P(w_i | \cdot) - \log Q(w_i | \cdot)),$$

ossia (a segno) la differenza di log-likelihood media. Per evitare falsi positivi, si usano test a ricampionamento (bootstrap o test appaiato per frase) e si riportano intervalli su  $\Delta H$ .

**Pitfall frequenti.** La  $PP$  non è confrontabile tra tokenizzazioni diverse o con policy di OOV diverse; cambiare  $N$  o l'uso di  $\langle UNK \rangle$  altera i valori. Il mismatch di dominio gonfia  $H$  indipendentemente dalla bontà del modello. Inoltre,  $PP$  bassa non garantisce miglioramento su task a valle: va sempre verificato l'effetto su metriche applicative.

### 3.2 Generalization and Zeros

Un modello a  $n$ -grammi stimato con massima verosimiglianza generalizza bene solo se le probabilità apprese non si limitano a riprodurre i conteggi del training (più è alto il numero di  $n$ -gram più si adatta specificatamente al testo) ma assegnano massa anche a sequenze plausibili mai osservate. Con MLE, per un bigram vale

$$\hat{P}_{\text{MLE}}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

per cui ogni  $n$ -gram mai visto ha probabilità 0. In una frase  $w_{1:N}$  basta un solo fattore nullo per annullare l'intera  $P(w_{1:N})$ , facendo esplodere la perplexity: se  $P(w_j | \cdot) = 0$  allora

$$\log P(w_{1:N}) = \sum_{i=1}^N \log P(w_i | \cdot) = -\infty, \quad \text{PP}(W) = \exp\left(-\frac{1}{N} \sum_i \log P\right) = +\infty$$

Questo è il *problema degli zeri*: uno stimatore perfetto sul training (**overfitting**) può essere pessimo sul test.

Gli  $n$ -grammi crescono esponenzialmente con  $n$  e con  $|V|$ ; anche con corpora estesi, lo spazio dei contesti è in gran parte inesplorato. Il modello MLE assegna tutta la massa agli eventi visti e nessuna a quelli non visti, violando il principio di *regolarità* richiesto dalla generalizzazione. Dal punto di vista informativo, minimizzare l'errore sul training non implica minimizzare l'errore di generalizzazione  $H(W_{\text{test}}; P)$ .

Per evitare zeri dovuti a parole sconosciute, si adotta una valutazione a **vocabolario chiuso**: si definisce un set  $V$  sul training e si mappano le forme rare/non viste in un token speciale  $\langle \text{UNK} \rangle$ . Durante l'addestramento si sostituiscono nel training tutte le parole con frequenza  $< \tau$  con  $\langle \text{UNK} \rangle$ , così il modello apprende probabilità per  $\langle \text{UNK} \rangle$ . In test, ogni parola fuori da  $V$  viene mappata a  $\langle \text{UNK} \rangle$  prima del calcolo di  $P(w_i | \cdot)$ .

**Dalla frequenza alla predizione: riservare massa agli eventi non visti** L'idea chiave per generalizzare è *spostare* parte della probabilità dagli eventi frequenti verso la coda, così che gli eventi non visti ricevano massa non nulla. Un esempio formale (che svilupperemo nelle sezioni sullo smoothing) è la famiglia di stimatori che modificano i conteggi  $c$  in conteggi "effettivi"  $c^*$  e poi applicano MLE:

$$\tilde{P}(w_i | h) = \frac{c^*(h, w_i)}{\sum_{w'} c^*(h, w')}, \quad h = w_{i-n+1:i-1}$$

Con questo schema, anche quando  $c(h, w) = 0$  possiamo avere  $c^*(h, w) > 0$  grazie alla redistribuzione di massa.

**Good-Turing (intuizione)** Una strategia classica stima la *riserva* di probabilità per gli eventi non visti a partire dalla frequenza degli eventi rari. Se  $N_c$  è il numero di  $n$ -gram che compaiono esattamente  $c$  volte nel training, l'idea di Good-Turing rimpiazza (per  $c > 0$ ) il conteggio con

$$c^* = \frac{(c+1) N_{c+1}}{N_c}, \quad \text{e assegna agli eventi mai visti } (c=0) \text{ una massa } p_0 = \frac{N_1}{N_{\text{tot}}}$$

con normalizzazione opportuna. In pratica si usa spesso in combinazione con backoff/interpolazione. (I dettagli e le correzioni di stima saranno trattati nello smoothing.)



### 3.3 Smoothing: Add-one (Laplace) smoothing

Lo **smoothing** evita probabilità nulle per  $n$ -grammi mai osservati e migliora la capacità di generalizzazione. La forma più semplice è l'**add-one (Laplace)**, che aggiunge una pseudo-occorrenza a ogni possibile continuazione del contesto.

Dato un contesto (*history*)  $h = w_{i-n+1:i-1}$ , il modello MLE assegna

$$\hat{P}_{\text{MLE}}(w | h) = \frac{c(h, w)}{c(h)}, \quad c(h) = \sum_{w'} c(h, w')$$

Con Laplace, sommiamo 1 al conteggio di ogni parola del vocabolario  $V$  (*closed vocabulary* che include anche  $\langle \text{UNK} \rangle$ ) e normalizziamo:

$$\tilde{P}_{\text{Lap}}(w | h) = \frac{c(h, w) + 1}{c(h) + |V|}$$

La normalizzazione è garantita perché  $\sum_w (c(h, w) + 1) = c(h) + |V|$ .

Con una visione bayesiana, l'add-one è un caso particolare della famiglia di **Lidstone** che usa un *prior Dirichlet simmetrico* con parametro  $\alpha > 0$ . La previsione *posterior predictive* è

$$\tilde{P}_{\alpha}(w | h) = \frac{c(h, w) + \alpha}{c(h) + \alpha|V|}, \quad \alpha = 1 \Rightarrow \text{Laplace}$$

Questa interpretazione chiarisce che lo smoothing equivale ad aggiungere  $\alpha$  pseudo-conteggi a *tutte* le uscite possibili dal contesto, prima di normalizzare.

**Ex.**

Supponiamo un bigram LM con contesto il corpus del Berkeley Restaurant

|         | i  | want | to  | eat | chinese | food | lunch | spend |
|---------|----|------|-----|-----|---------|------|-------|-------|
| i       | 6  | 828  | 1   | 10  | 1       | 1    | 1     | 3     |
| want    | 3  | 1    | 609 | 2   | 7       | 7    | 6     | 2     |
| to      | 3  | 1    | 5   | 687 | 3       | 1    | 7     | 212   |
| eat     | 1  | 1    | 3   | 1   | 17      | 3    | 43    | 1     |
| chinese | 2  | 1    | 1   | 1   | 83      | 2    | 1     | 1     |
| food    | 16 | 1    | 16  | 1   | 2       | 5    | 1     | 1     |
| lunch   | 3  | 1    | 1   | 1   | 1       | 2    | 1     | 1     |
| spend   | 2  | 1    | 2   | 1   | 1       | 1    | 1     | 1     |

$$P^*(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

|         | i       | want    | to      | eat     | chinese | food    | lunch   | spend   |
|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| i       | 0.0015  | 0.21    | 0.00025 | 0.0025  | 0.00025 | 0.00025 | 0.00025 | 0.00075 |
| want    | 0.0013  | 0.00042 | 0.26    | 0.00084 | 0.0029  | 0.00025 | 0.0025  | 0.00084 |
| to      | 0.00078 | 0.00026 | 0.0013  | 0.18    | 0.00078 | 0.00026 | 0.0018  | 0.055   |
| eat     | 0.00046 | 0.00046 | 0.0014  | 0.00046 | 0.0078  | 0.0014  | 0.02    | 0.00046 |
| chinese | 0.0012  | 0.00062 | 0.00062 | 0.00062 | 0.052   | 0.00062 | 0.0012  | 0.00062 |
| food    | 0.0063  | 0.00039 | 0.0063  | 0.00039 | 0.00079 | 0.002   | 0.00039 | 0.00039 |
| lunch   | 0.0017  | 0.00056 | 0.00056 | 0.00056 | 0.00056 | 0.0011  | 0.00056 | 0.00056 |
| spend   | 0.0012  | 0.00058 | 0.0012  | 0.00058 | 0.00058 | 0.00058 | 0.00058 | 0.00058 |

$$\begin{aligned}
c^*(w_{n-1}w_n) &= P^*(w_{n-1}w_n)N \\
&= P^*(w_n|w_{n-1})P(w_{n-1})N \\
&= P^*(w_n|w_{n-1})C(w_{n-1}) \\
&= \frac{[C(w_{n-1}w_n) + 1]C(w_{n-1})}{C(w_{n-1}) + V}
\end{aligned}$$

|         | i    | want  | to    | eat   | chinese | food | lunch | spend |
|---------|------|-------|-------|-------|---------|------|-------|-------|
| i       | 3.8  | 527   | 0.64  | 6.4   | 0.64    | 0.64 | 0.64  | 1.9   |
| want    | 1.2  | 0.39  | 238   | 0.78  | 2.7     | 2.7  | 2.3   | 0.78  |
| to      | 1.9  | 0.63  | 3.1   | 430   | 1.9     | 0.63 | 4.4   | 133   |
| eat     | 0.34 | 0.34  | 1     | 0.34  | 5.8     | 1    | 15    | 0.34  |
| chinese | 0.2  | 0.098 | 0.098 | 0.098 | 0.098   | 8.2  | 0.2   | 0.098 |
| food    | 6.9  | 0.43  | 6.9   | 0.43  | 0.86    | 2.2  | 0.43  | 0.43  |
| lunch   | 0.57 | 0.19  | 0.19  | 0.19  | 0.19    | 0.38 | 0.19  | 0.19  |
| spend   | 0.32 | 0.16  | 0.32  | 0.16  | 0.16    | 0.16 | 0.16  | 0.16  |

Tutte le uscite non viste ricevono massa non nulla e la distribuzione resta normalizzata.

L'add-one elimina gli zeri e stabilizza la cross-entropy di test  $H(W; P) = -\frac{1}{N} \sum_i \log \tilde{P}(w_i | h_i)$ , evitando  $+\infty$  nella perplexity. Tuttavia, tende a *sovrastimare* gli eventi non visti e a *sottostimare* quelli frequenti, soprattutto quando  $|V|$  è grande e  $c(h)$  è piccolo. **In pratica questo comporta perplexity peggiori rispetto a metodi più informati.** Un rimedio parziale è usare  $\alpha < 1$  (Lidstone), ma la stima resta grossolana se paragonata a Kneser–Ney.

**Nota:** In valutazione a vocabolario chiuso,  $|V|$  deve includere  $\langle UNK \rangle$ ,  $\langle s \rangle$  e  $\langle /s \rangle$  se presenti nel training. L'add-one assegna automaticamente probabilità positive anche a  $\langle UNK \rangle$ , consentendo di calcolare  $P(w_{1:N}) = \prod_i \tilde{P}(w_i | h_i)$  senza zeri.

**Nota:** L'add-one è utile come baseline e come strumento didattico: è semplice da implementare, completamente deterministico e rende trasparente il ruolo di  $|V|$  e dei conteggi. Per applicazioni serie su bigrammi/trigrammi è preferibile adottare smoothing con *discount* e backoff/interpolazione (es. Kneser–Ney), ottimizzando i parametri su validation.

### 3.4 Interpolation, Backoff, and Web-Scale LMs

In un LM interpolato combiniamo più modelli (ad es. uni/bi/trigram) con pesi non negativi che sommano a 1. Per un contesto  $h$  e la prossima parola  $w$  scriviamo

$$\tilde{P}(w | h) = \sum_{m=1}^M \lambda_m P_m(w | h), \quad \lambda_m \geq 0, \quad \sum_{m=1}^M \lambda_m = 1$$

con  $P_m$  stime (MLE o smussate) a diversi ordini o da diversi domini. L'obiettivo è scegliere  $\lambda$  per massimizzare la log-likelihood su un *dev set*  $\mathcal{D} = \{(h_i, w_i)\}_{i=1}^N$ :

$$\mathcal{L}(\lambda) = \sum_{i=1}^N \log \tilde{P}(w_i | h_i) = \sum_{i=1}^N \log \left( \sum_{m=1}^M \lambda_m P_m(w_i | h_i) \right).$$

Questa è la **linear interpolation (Jelinek–Mercer)** con pesi *globali*. Estensioni *context-dependent* sostituiscono  $\lambda_m$  con  $\lambda_m(h)$ , vincolati a sommare a 1 per ogni  $h$ .

#### 3.4.1 Linear Interpolation

Il modello interpolato è una *mistura* in cui, per ogni istanza  $(h_i, w_i)$ , una componente latente  $z_i \in \{1, \dots, M\}$  “sceglie” quale modello  $P_m$  ha generato  $w_i$ . Introdurre  $z_i$  consente di massimizzare  $\mathcal{L}(\lambda)$  con **Expectation-Maximization (EM)** sotto il vincolo  $\sum_m \lambda_m = 1$ .

**E-step.** Calcoliamo le responsabilità (posteriori) di ciascuna componente sul dato  $i$ :

$$\gamma_i(m) = P(z_i=m | h_i, w_i) = \frac{\lambda_m P_m(w_i | h_i)}{\sum_{j=1}^M \lambda_j P_j(w_i | h_i)}$$

**M-step.** Aggiorniamo i pesi come frazioni attese di assegnazioni, con normalizzazione:

$$\lambda_m^{\text{new}} = \frac{1}{N} \sum_{i=1}^N \gamma_i(m), \quad (\text{e quindi } \sum_m \lambda_m^{\text{new}} = 1)$$

Questi passi aumentano monotonamente  $\mathcal{L}(\lambda)$  e convergono a un ottimo locale. Per evitare degenerazioni quando alcune  $P_m(w | h)$  sono mai nulle, si usa smoothing nelle  $P_m$  o un piccolo termine di *flooring*. L'algoritmo EM nasce in contesti di *mixture models*. Nel classico **Gaussian Mixture Model (GMM)** su vettori  $x_i \in \mathbb{R}^d$  con  $M$  componenti, il modello è

$$P(x_i) = \sum_{m=1}^M \pi_m \mathcal{N}(x_i; \mu_m, \Sigma_m), \quad \sum_m \pi_m = 1$$

L'EM alterna:

**E-step.**

$$\gamma_i(m) = \frac{\pi_m \mathcal{N}(x_i; \mu_m, \Sigma_m)}{\sum_j \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}$$

**M-step.**

$$\begin{aligned} \pi_m &\leftarrow \frac{1}{N} \sum_i \gamma_i(m), & \mu_m &\leftarrow \frac{\sum_i \gamma_i(m) x_i}{\sum_i \gamma_i(m)}, \\ \Sigma_m &\leftarrow \frac{\sum_i \gamma_i(m) (x_i - \mu_m)(x_i - \mu_m)^\top}{\sum_i \gamma_i(m)} \end{aligned}$$

Nel nostro caso, le componenti non sono Gaussiane ma *discrete*  $P_m(w | h)$ ; tuttavia la struttura dell'EM è identica e fornisce un criterio principled per impostare i pesi  $\lambda$ .

### 3.4.2 Kneser–Ney Interpolation

L'interpolazione di **Kneser–Ney (KN)** è uno smoothing allo stato dell'arte per  $n$ -grammi. Combina *discount assoluto* sul livello alto con una **distribuzione di backoff** specializzata che misura la *diversità dei contesti* di una parola, non solo la sua frequenza grezza. Questo migliora drasticamente le code (parole comuni come continuazioni "generiche" ma non come inizi di nuove collocazioni).

Per un contesto  $h$  e parola  $w$ , lo **bigram KN** è:

$$P_{\text{KN}}(w | h) = \frac{\max(c(h, w) - D, 0)}{c(h)} + \lambda(h) P_{\text{cont}}(w)$$

con  $D \in (0, 1)$  sconto assoluto e

$$\lambda(h) = \frac{D N_{1+}(h*)}{c(h)}, \quad P_{\text{cont}}(w) = \frac{N_{1+}(*w)}{N_{1+}(**)}.$$

Qui  $N_{1+}(h*)$  è il numero di tipi distinti che seguono  $h$  almeno una volta;  $N_{1+}(*w)$  è il numero di contesti distinti che precedono  $w$ ;  $N_{1+}(**)$  è il numero totale di bigrammi distinti osservati. L'idea è che la probabilità di backoff non è l'unigramma  $\hat{P}(w) = c(w) / \sum_{w'} c(w')$ , bensì la *probabilità di continuazione*  $P_{\text{cont}}(w)$ : parole che appaiono in molti contesti diversi ottengono più massa.

Per **trigrammi** si interpola ricorsivamente con il KN del livello inferiore:

$$P_{\text{KN}}(w | u, v) = \frac{\max(c(uv, w) - D, 0)}{c(uv)} + \lambda(uv) P_{\text{KN}}(w | v)$$

con  $\lambda(uv) = \frac{D N_{1+}(uv*)}{c(uv)}$  e il termine di backoff  $P_{\text{KN}}(w | v)$  definito a sua volta come nel bigram KN.

Per ordini superiori si procede allo stesso modo.

Lo **Modified Kneser–Ney** usa sconti diversi a seconda del conteggio di cella:  $D_1$  per  $c = 1$ ,  $D_2$  per  $c = 2$ ,  $D_{3+}$  per  $c \geq 3$ . La forma trigramma diventa

$$P_{\text{MKN}}(w | u, v) = \frac{\max(c(uv, w) - D(c(uv, w)), 0)}{c(uv)} + \lambda(uv) P_{\text{MKN}}(w | v)$$

con

$$\lambda(uv) = \frac{D_1 N_1(uv*) + D_2 N_2(uv*) + D_{3+} N_{3+}(uv*)}{c(uv)}$$

ove  $N_r(uv*)$  è il numero di tipi distinti che seguono  $uv$  esattamente  $r$  volte. Gli sconti si stimano dai *conteggi di conteggi*  $n_r$  (numero di  $n$ -gram che compaiono  $r$  volte) su un dev set. Una stima comune (Chen & Goodman) è:

$$D_1 = 1 - \frac{2n_2}{n_1}, \quad D_2 = 2 - \frac{3n_3}{n_2}, \quad D_{3+} = 3 - \frac{4n_4}{n_3}$$

con clipping per garantire  $D_r \in (0, 1)$ .

La chiave di KN è il termine di backoff basato su *diversità di contesti*. Parole molto frequenti ma collocate in pochi contesti (ad es. *New* prima di *York*) non ricevono troppa massa nel livello basso, mentre parole che possono continuare molti contesti diversi ricevono un prior più alto. Questo riduce *over-allocation* a token frequentissimi e migliora la predizione delle continuazioni rare ma plausibili.

Il fattore  $\lambda(h)$  è scelto in modo che la probabilità totale sommata su  $w$  sia 1. Infatti, la massa scontata  $\sum_{w: c(h, w) > 0} \frac{D}{c(h)} = \frac{D N_{1+}(h*)}{c(h)}$  viene interamente riallocata al modello di ordine inferiore tramite  $\lambda(h)$ , e  $P_{\text{cont}}(\cdot)$  è già normalizzato per costruzione.

### 3.4.3 Web-scale LMs

Su corpora web-scale (decine di miliardi di token) lo stoccaggio e l'inferenza con modelli normalizzati sono costosi. Lo **stupid backoff** sacrifica la normalizzazione per semplicità e velocità. Per i trigrammi:

$$S(w \mid h) = \begin{cases} \frac{c(h, w)}{c(h)} & \text{se } c(h, w) > 0, \\ \gamma S(w \mid h') & \text{se } c(h, w) = 0, \end{cases}$$

con  $0 < \gamma < 1$  (tipicamente  $\gamma \approx 0.4$ ) e  $S$  non è una probabilità vera: non somma a 1 su  $w$ . In ranking e decoding, tuttavia, confrontare i punteggi  $S(\cdot \mid \cdot)$  è spesso sufficiente, e la mancanza di normalizzazione riduce enormemente costi di calcolo e implementazione. In pratica si combinano: conteggi massivi (MapReduce), pruning aggressivo degli  $n$ -grammi a basso conteggio, tries compressi e caching.

In domini “ordinari” i modelli interpolati/backoff normalizzati (es. interpolated Kneser–Ney) tendono a fornire la migliore perplexity. Su scala web o in sistemi di ranking real-time, lo stupid backoff è competitivo perché riduce la latenza. In tutti i casi, i parametri ( $\lambda$ , sconti,  $\gamma$ ) si selezionano su *dev* minimizzando  $H(W_{\text{dev}}; P) = -\frac{1}{N} \sum_i \log \tilde{P}(w_i \mid \cdot)$  o massimizzando la metrica applicativa.

## 4 POS & NER

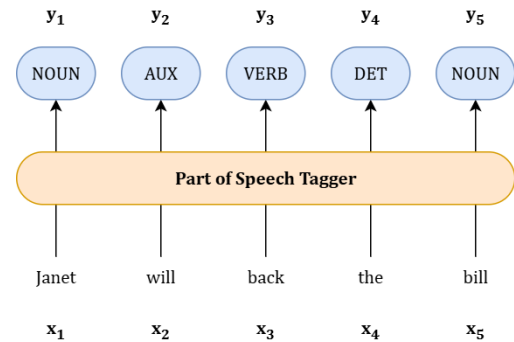
### 4.1 Part-of-Speech (POS) Tagging

Il *Part-of-Speech tagging* assegna a ciascun token  $w_i$  la sua categoria grammaticale  $y_i$  (nome, verbo, aggettivo, ecc.), data la frase  $w_{1:n}$ . Formalmente è un problema di **sequence labeling**: vogliamo la sequenza di etichette più probabile

$$\hat{y}_{1:n} = \arg \max_{y_{1:n}} P(y_{1:n} | w_{1:n})$$

con vincoli di coerenza tra tag adiacenti (ad es. un determinante tende a precedere un nome) e dipendenza dal contesto.

Il POS facilita l'analisi morfosintattica, la disambiguazione lessicale e step successivi (lemmatizzazione, parsing, NER). Un buon tagger riduce ambiguità come in: “*La porta è chiusa*” (PORTA/NOUN) vs. “*Porta il libro*” (PORTA/VERB), o “*banca*” (istituto, NOUN) vs. “*banca a sinistra*” (VERB colloquiale in alcuni registri), e gestisce clitici: “*gliel’ho detto*”.



#### 4.1.1 Closed-Class vs. Open-Class Words

Nel lessico distinguiamo due famiglie con comportamento molto diverso: le *parole chiuse* (*closed class*) e le *parole aperte* (*open class*). La differenza non è solo tassonomica: influisce su frequenze, ambiguità, produttività morfologica e sulle strategie di tagging.

- Le **parole chiuse** appartengono a insiemi piccoli e relativamente stabili, con inventario quasi fisso nella lingua standard. Comprendono determinanti, preposizioni, congiunzioni, pronomi, ausiliari, particelle, negazioni, alcuni avverbi funzionali e la punteggiatura. Il loro ruolo è soprattutto *grammaticale*: stabiliscono relazioni sintattiche e marcano funzioni.
- Le **parole aperte** formano classi a inventario non limitato e in continua crescita: nomi comuni e propri, verbi lessicali, aggettivi qualificativi, molti avverbi derivati e simboli tecnici. Il loro ruolo è *lessicale/semantico*: introducono contenuto informativo e si arricchiscono con neologismi, prestiti, nomi propri.

Nel corpus, le parole chiuse sono altissime in frequenza ma poche come tipi distinti; contribuiscono alla testa della distribuzione (legge di Zipf). Le parole aperte hanno frequenza individuale più bassa e una *coda lunga* di tipi rari; dominano la crescita del vocabolario secondo Heaps. Se  $N$  è il numero di token e  $|V|$  il vocabolario, la dinamica delle aperte segue spesso  $|V_{\text{open}}(N)| \approx K N^\beta$  con  $0 < \beta < 1$ , mentre  $|V_{\text{closed}}|$  rimane pressoché costante. La produttività morfologica è elevata nelle classi aperte (derivazione e flessione), mentre nelle chiuse è limitata o nulla.

Per le classi chiuse il tagging sfrutta forti indizi lessicali: una tabella di frequenza parola  $\rightarrow$  tag è spesso sufficiente e l'ambiguità è bassa (ad es. *di* è quasi sempre ADP). Per le classi aperte l'ambiguità è alta (*porta* NOUN/VERB) e la disambiguazione richiede contesto sintagmatico e morfologia. Inoltre, gli OOV nascono prevalentemente nelle classi aperte; per questo gli embedding subword e la gestione di suffissi/prefissi aiutano soprattutto su nomi, verbi e aggettivi.

Adottiamo le classi **Universal Dependencies (UPOS)**:

|                     | Tag   | Descrizione         | Esempio IT                    |
|---------------------|-------|---------------------|-------------------------------|
| <b>Open Class</b>   | ADJ   | aggettivo           | <i>nuovo, bello</i>           |
|                     | ADV   | avverbio            | <i>molto, sempre</i>          |
|                     | NOUN  | nome comune         | <i>porta, libro</i>           |
|                     | VERB  | verbo lessicale     | <i>portare, studiare</i>      |
|                     | PROPN | nome proprio        | <i>Roma, San_Francesco</i>    |
|                     | INTJ  | interiezione        | <i>oh, eh</i>                 |
| <b>Closed Class</b> | ADP   | preposizione        | <i>di, a, con</i>             |
|                     | AUX   | ausiliare           | <i>essere, avere (aus.)</i>   |
|                     | CCONJ | congiunzione coord. | <i>e, ma, o</i>               |
|                     | DET   | determinante        | <i>il, una, questo</i>        |
|                     | NUM   | numero              | <i>due, 100</i>               |
|                     | PART  | particella          | <i>non, ci (part.)</i>        |
|                     | PRON  | pronome             | <i>io, gli, che</i>           |
|                     | SCONJ | congiunzione sott.  | <i>che, perché</i>            |
| <b>Other</b>        | PUNCT | punteggiatura       | <i>, . —</i>                  |
|                     | SYM   | simbolo             | <i>€, %</i>                   |
|                     | X     | altro               | <i>stringhe miste, codice</i> |

La grana fine (morfologia: genere/numero, tempo/modo/persona) si esprime con feat addizionali e talvolta con un tagset più dettagliato (*XPOS*) quando serve compatibilità con risorse legacy.

Molte forme sono *ambiguous* (es. “porta” NOUN/VERB). La disambiguazione sfrutta il contesto:  $P(y_i | y_{i-1}, w_{1:n})$  differisce se il token precedente è un determinante, un pronome o una preposizione. Decisioni di pre-processing influenzano il tagging: separare “l” da *ho* aiuta a assegnare DET e AUX corretti; trattare MWE come *New\_York* evita confondere PROPN con due token separati.

La metrica standard è l’**accuracy** per token:

$$\text{Acc} = \frac{\# \text{ token con tag corretto}}{\# \text{ token totali}}$$

Con etichette esclusive per token, l’accuracy coincide con la micro- $F_1$ . È buona prassi riportare anche errori per classe (confusioni tipiche: AUX/VERB, ADJ/NOUN) e analizzare le prestazioni su *OOV* e domini diversi (news vs. social). Errori di tokenizzazione o di normalizzazione propagano a valle e vanno contabilizzati.

Su corpora bilanciati e ben tokenizzati, i tagger neurali moderni in italiano e lingue europee raggiungono tipicamente  $\text{Acc} \geq 97\%$  con encoder pre-addestrati; CRF e HMM di buona qualità si collocano poco sotto. In domini rumorosi o con forte variazione morfologica l’accuracy cala sensibilmente: la gestione di OOV, clitici e varianti ortografiche incide molto.

## 4.2 Named Entity Recognition (NER)

La *Named Entity Recognition* identifica e classifica menzioni di entità nel testo (persone, organizzazioni, luoghi, date, quantità, ecc.). Data una sequenza di token  $w_{1:n}$ , l'obiettivo è produrre una sequenza di etichette  $y_{1:n}$  che segmenti le menzioni e ne assegni il tipo. È un problema di **sequence labeling** con vincoli strutturali sui confini e sulla coerenza delle etichette.

Si rappresentano le menzioni come segmenti contigui con schemi standard:

- **BIO**: **B- TYPE** (inizio), **I- TYPE** (interno), **O** (fuori da entità).
- **BIOES**: aggiunge **E- TYPE** (fine) e **S- TYPE** (menzione di un solo token) per rendere più espliciti i confini.

Ex.

| Words      | IO Label | BIO Label | BIOES Label |
|------------|----------|-----------|-------------|
| Jane       | I-PER    | B-PER     | B-PER       |
| Villanueva | I-PER    | I-PER     | E-PER       |
| of         | O        | O         | O           |
| United     | I-ORG    | B-ORG     | B-ORG       |
| Airlines   | I-ORG    | I-ORG     | I-ORG       |
| Holding    | I-ORG    | I-ORG     | E-ORG       |
| discussed  | O        | O         | O           |
| the        | O        | O         | O           |
| Chicago    | I-LOC    | B-LOC     | S-LOC       |
| route      | O        | O         | O           |
| .          | O        | O         | O           |

La metrica standard valuta *menzioni complete* (span + tipo). Si computano precision, recall e  $F_1$  su insiemi di menzioni predette ( $\hat{\mathcal{E}}$ ) e gold ( $\mathcal{E}$ ):

$$\text{Prec} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}|}{|\hat{\mathcal{E}}|}, \quad \text{Rec} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}|}{|\mathcal{E}|}, \quad F_1 = \frac{2 \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$$

Un match è corretto se *entrambi* i confini e il tipo coincidono. È buona prassi riportare anche i risultati per classe (PER/ORG/LOC/...) e distinguere micro e macro-averaging.

Tokenizzazione e normalizzazione coerenti sono cruciali: menzioni composte (*San Francesco*), apostrofi e accenti influiscono sui confini. Le **gazetteer** (liste di nomi propri, toponimi, sigle) sono utili come feature ma vanno gestite con cura per evitare overfitting di dominio. La rappresentazione a caratteri/subword mitiga gli OOV e cattura regolarità ortografiche (suffissi aziendali, notazioni alfanumeriche).

Nomi comuni vs. propri (*apple* vs. *Apple*), metonimie (*Roma* città vs. squadra), entità annidate o sovrapposte (*Università di [B-ORG Roma [I-LOC Tre]]*), e sigle creative richiedono bias di decodifica e/o schemi di annotazione dedicati (NER piatto vs. NER annidato). In corpora standard, le menzioni sono *non sovrapposte* e il modello produce una sola etichetta per token.

Su corpora ben bilanciati e in lingua standard, modelli neurali con encoder pre-addestrati e CRF raggiungono spesso  $F_1 \geq 90\%$  a livello di entità per classi frequenti (PER/ORG/LOC). Il calo è più marcato su classi rare (MISC, eventi) o in domini rumorosi (social, OCR). Errori tipici: confini sballati di un token (B-/E-), tipo corretto ma span errato, o tipo confuso tra ORG/LOC per toponimi istituzionali.



### 4.3 Standard Algorithms for NER/POS Tagging

Dato un input  $w_{1:n}$  (token), vogliamo la sequenza di etichette  $y_{1:n}$  che massimizza la probabilità condizionata o congiunta a seconda del modello:

$$\hat{y}_{1:n} = \arg \max_{y_{1:n}} P(y_{1:n} | w_{1:n}) \quad (\text{discriminativo}) \quad \text{o} \quad \hat{y}_{1:n} = \arg \max_{y_{1:n}} P(y_{1:n}, w_{1:n}) \quad (\text{generativo}).$$

Alcune famiglie di modelli supervisionati per l'etichettatura di sequenza sono

#### 4.3.1 Hidden Markov Models

Un **HMM** è un modello generativo per l'etichettatura di sequenza. Assume una catena di stati latenti  $y_{1:n}$  (i tag) che genera i token osservati  $w_{1:n}$  con emissioni condizionate dallo stato corrente.

- transizioni tra tag  $a_{y',y} = P(y_i=y | y_{i-1}=y')$ ;
- emissioni  $b_y(w) = P(w_i=w | y_i=y)$ .

La congiunta fattorizza come

$$P(y_{1:n}, w_{1:n}) = \pi_{y_1} b_{y_1}(w_1) \prod_{i=2}^n a_{y_{i-1}, y_i} b_{y_i}(w_i).$$

In pratica si aggiungono stati speciali  $\langle s \rangle$  e  $\langle /s \rangle$  per inizio/fine frase.

**Codifica di Viterbi.** Dato  $w_{1:n}$ , cerchiamo la sequenza di tag a massima verosimiglianza *a posteriori* (MAP):

$$\hat{y}_{1:n} = \arg \max_{y_{1:n}} P(y_{1:n} | w_{1:n}) = \arg \max_{y_{1:n}} P(y_{1:n}, w_{1:n}).$$

La cui **ricorrenza di Viterbi** in log-spazio è

$$\delta_i(y) = \max_{y'} \left\{ \delta_{i-1}(y') + \log a_{y',y} \right\} + \log b_y(w_i),$$

$$\delta_1(y) = \log \pi_y + \log b_y(w_1),$$

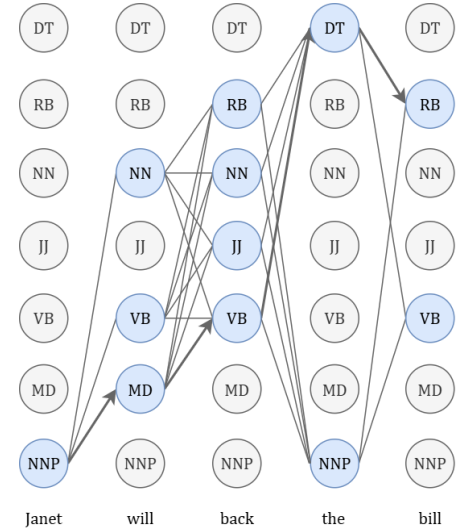
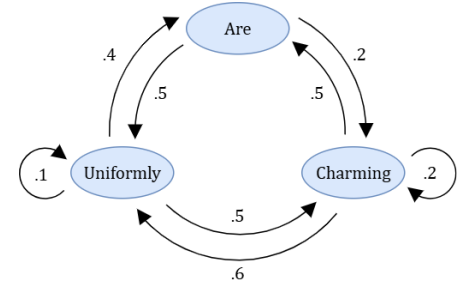
con backpointer  $\psi_i(y) = \arg \max_{y'} \{ \delta_{i-1}(y') + \log a_{y',y} \}$  e ricostruzione all'indietro a partire da  $\hat{y}_n = \arg \max_y \delta_n(y)$ .

**Addestramento supervisionato (MLE).** Con training etichettato, le stime di massima verosimiglianza sono basate sui conteggi:

$$\hat{\pi}_y = \frac{c(y \text{ in posizione } 1)}{\sum_{y'} c(y' \text{ in posizione } 1)}, \quad \hat{a}_{y',y} = \frac{c(y' \rightarrow y)}{\sum_{y''} c(y' \rightarrow y'')}, \quad \hat{b}_y(w) = \frac{c_y(w)}{\sum_{w'} c_y(w')},$$

con  $c(y' \rightarrow y)$  conteggi di transizioni adiacenti nel corpus e  $c_y(w)$  conteggi di emissione della parola  $w$  dal tag  $y$ .

In POS l'emissione è tipicamente parola  $\rightarrow$  tag; in NER si può applicare alle etichette BIO/BIOES. Gli HMM sono competitivi come baseline, specialmente con pochi dati, e beneficiano molto di: (i) tokenizzazione coerente, (ii) gestione di MWE per nomi propri, (iii) smoothing mirato sulle emissioni (classe aperta) e dizionari per le classi chiuse.



### 4.3.2 Conditional Random Fields (CRF)

Un **CRF a catena lineare** modella direttamente la probabilità condizionata di una sequenza di etichette  $y_{1:n}$  dato l'input  $x_{1:n}$  (token, più eventuali feature), combinando molteplici indizi locali in modo *globale* sull'intera frase.

Sia  $\phi_k(y_{i-1}, y_i, x, i)$  una funzione di feature locale (di solito binaria: 1 se la regola vale, 0 altrimenti) che dipende solo dalla *coppia di tag adiacenti*  $(y_{i-1}, y_i)$ , dalla *posizione*  $i$  e dall'*input della frase*  $x$ . Con pesi  $\theta_k$ , la probabilità è

$$P_\theta(y | x) = \frac{1}{Z_\theta(x)} \exp \left( \sum_{i=1}^n \sum_k \theta_k \phi_k(y_{i-1}, y_i, x, i) \right),$$

con partizione (*normalizzatore*)

$$Z_\theta(x) = \sum_{y' \in \mathcal{Y}^n} \exp \left( \sum_{i=1}^n \sum_k \theta_k \phi_k(y'_{i-1}, y'_i, x, i) \right).$$

**Nota:**  $Z_\theta(x)$  dipende solo da  $x$  e  $\theta$ , quindi non influisce sull' $\arg \max_y$  in decodifica.

Convenientemente si raggruppano le feature in *potenziali* di transizione e di stato:

$$\text{score}(y | x) = \sum_{i=1}^n (s_i(y_i, x) + t_i(y_{i-1}, y_i, x)),$$

con  $s_i$  funzioni sul tag corrente e  $t_i$  funzioni sulla coppia  $(y_{i-1}, y_i)$ . Allora  $P_\theta(y | x) \propto \exp(\text{score}(y | x))$ .

**Decodifica di Viterbi.** La sequenza più probabile si ottiene massimizzando la somma degli score:

$$\hat{y}_{1:n} = \arg \max_{y \in \mathcal{Y}^n} \sum_{i=1}^n (s_i(y_i, x) + t_i(y_{i-1}, y_i, x)),$$

che si risolve con **Viterbi** usando come emissioni/transizioni i punteggi  $s_i$  e  $t_i$  calcolati dalle feature del CRF.

**Addestramento Supervisionato(MLE).** Dato un dataset annotato  $\{(x^{(j)}, y^{(j)})\}_{j=1}^M$ , si massimizza la log-verosimiglianza regolarizzata

$$\mathcal{L}(\theta) = \sum_{j=1}^M \left[ \sum_{i,k} \theta_k \phi_k(y_{i-1}^{(j)}, y_i^{(j)}, x^{(j)}, i) - \log Z_\theta(x^{(j)}) \right] - \frac{\lambda}{2} \|\theta\|_2^2.$$

Il gradiente è la differenza tra *conteggi empirici* e *conteggi attesi sotto il modello*:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \sum_{j,i} \phi_k(y_{i-1}^{(j)}, y_i^{(j)}, x^{(j)}, i) - \sum_j \sum_y P_\theta(y | x^{(j)}) \sum_i \phi_k(y_{i-1}, y_i, x^{(j)}, i).$$

I termini attesi si calcolano in  $\mathcal{O}(n|\mathcal{Y}|^2)$  con **forward-backward**. In pratica si usa ottimizzazione numerica (SGD/Adam, L-BFGS) con *cross-entropy* e regolarizzazione.

Le feature locali sono indicatori binari costruiti da **template** astratti riempiti dai valori osservati. Esempi tipici per POS/NER:

- **Lessico/forma:** parola, minuscola/maiuscola, cifre, punteggiatura, *wordshape* (p.es.  $Xx\_xx, xx$ ).
- **Contesto:** token a  $\pm 1, \pm 2$  posizioni; bigrammi di tag  $(y_{i-1}, y_i)$ .
- **Morfologia:** prefissi/suffissi di lunghezza 1–4; segnalazione di apostrofi/clitici.
- **Gazetteer:** appartenenza a liste (toponimi, organizzazioni), con attenzione all’overfitting di dominio.
- **Vincoli BIO/BIOES** (per NER): proibizioni di transizioni illegali (es. **I-L0C** non può seguire **0**).

Tutte possono essere codificate come binarie; feature numeriche si includono come valori reali.

A differenza di un softmax token-wise (MaxEnt), il CRF *normalizza globalmente* sulla sequenza, modellando dipendenze tra tag adiacenti e riducendo incoerenze (es. sequenze BIO impossibili). Rispetto a HMM, consente *feature arbitrarie* sull’osservazione senza dover espandere lo spazio degli stati.

Oltre alle due di modelli appena descritte, abbiamo inoltre

- **Neural sequence models (RNN/BiLSTM/Transformer):** Qui le rappresentazioni si imparano dai dati. Si usano embedding a parola/subword e un codificatore contestuale, ad esempio

$$\mathbf{h}_{1:n} = \text{BiLSTM}(\text{emb}(w_{1:n})) \quad \text{oppure} \quad \mathbf{H} = \text{Transformer}(w_{1:n}),$$

seguiti da uno strato di classificazione token-wise (softmax) o un CRF in uscita per decodifica strutturata. Con encoder pre-addestrati, questi modelli riducono la necessità di feature manuali e gestiscono bene OOV e variazioni di dominio.

- **Large Language Models (LLM) fine-tunati:** Encoder pre-addestrati (es. BERT e varianti) si usano come backbone e si fine-tunano per POS/NER con una testa di classificazione (spesso con CRF). Il fine-tuning su un set annotato fornisce prestazioni allo stato dell’arte con minima ingegneria delle feature.

Tutti i modelli richiedono un **training set etichettato a mano** e sfruttano le stesse fonti informative: nei modelli "classici" tramite *feature umane* (morfologia, maiuscole, liste), nei modelli neurali tramite *representation learning*. Su inglese standard, a parità di dati e pre-processing, le prestazioni sono *simili* (POS  $\approx 97\%$  di accuracy; NER con  $F_1$  elevati sulle classi frequenti), con differenze dovute soprattutto a dominio, tokenizzazione e gestione di OOV.

La scelta del miglior modello può dipendere da vari fattori:

*Pochi dati o necessità di vincoli strutturali espliciti:* CRF (magari con livello a caratteri) è una scelta robusta.

*Dati moderati/ampi o dominio eterogeneo:* modelli neurali con encoder pre-addestrato (BiLSTM/Transformer) e, se serve, CRF in uscita.

*Baseline/risorse minime:* HMM fornisce una base semplice e riproducibile.

## 5 Vectorial Semantic and Embeddings

Nei modelli di n-gram o di classificazione testuale, le parole erano indici o stringhe prive di contenuto semantico. Qui introduciamo l'idea che il *significato* di una parola non coincide con l'etichetta simbolica, ma con il suo *uso* e le relazioni che intrattiene con altre parole e con i contesti in cui appare (Wittgenstein; Harris). Questo porta ai **desiderata** per una teoria del significato: distinguere sensi, catturare relazioni (sinonimia, antonimia, similarità, relatedness), gestire connotazioni e uso nel contesto.

### 5.1 Definitions

Un **lemma** raggruppa più **wordform** flesse; ciascuna voce può essere **polisemica** (più sensi). Esempi classici mostrano lemmi con sensi tecnici e comuni (come *mouse*: roditore vs. dispositivo d'ingresso). Il compito computazionale è spesso formulato come stima del senso più probabile in contesto:  $\hat{s} = \arg \max_s P(s | w, C)$ .

I **sinonimi** condividono il significato in alcuni contesti (*couch/sofa*, *automobile/car*); ma la sinonimia perfetta è rara: differiscono per registro, polisemia, sfumature pragmatiche. Il **principio linguistico del contrasto** ricorda che differenze formali tendono a riflettere differenze di significato, scoraggiando l'idea di sinonimia assoluta.

La **Similarità** misura quanto due parole condividano elementi di significato (*cow/horse*); **relatedness** (o associazione) misura relazioni anche non somiglianti (*coffee/cup*). Un criterio operativo per la similarità in modelli vettoriali è il **coseno** tra vettori semantici:

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|} = \frac{\sum_i v_i w_i}{\sqrt{\sum_i v_i^2} \sqrt{\sum_i w_i^2}}.$$

Il coseno è compreso tra 0 e 1 se i valori sono non negativi; normalizza per la lunghezza del vettore, correggendo il bias del prodotto scalare grezzo verso parole molto frequenti.

Un **campo semantico** raccoglie parole legate da uno stesso dominio e relazioni strutturate (*ristoranti: waiter, menu, chef, ...*). Il **topic modeling** (LDA) li apprende in modo non supervisionato: ogni documento è una mistura di topic, ciascun topic è una distribuzione su parole; i topic spiegano co-occorrenze senza introdurre sensi.

Parole che compaiono in contesti simili tendono ad avere significati simili (Harris). La semantica vettoriale *istanza* questa ipotesi costruendo rappresentazioni da co-occorrenze: matrici **term-document** o **term-term**, pesate e confrontate con misure come il coseno.

L'**antonimia** è un'opposizione rispetto a una singola dimensione semantica, pur mantenendo forte affinità lessicale (*hot/cold*, *up/down*, *rise/fall*). Gli antonimi possono definire una opposizione binaria o gli estremi di una scala gradabile; esistono anche casi *reversivi* (*entrare/uscire*). Nota pratica: negli spazi distribuzionali gli antonimi risultano spesso *vicini* perché condividono contesti; distinguere "opposto" da "simile" richiede segnali aggiuntivi (lessici etichettati, vincoli direzionali, ecc.).

Nel trattamento automatico del linguaggio naturale, i modelli statistici e neurali moderni (come i Transformer) operano su rappresentazioni numeriche delle parole, ma dietro molti sistemi di NLP classici e ibridi vi sono ancora risorse simboliche — cioè basi di conoscenza linguistiche esplicite, costruite manualmente da linguisti o semi-automaticamente. Tra queste, **WordNet** e **FrameNet** rappresentano due pilastri.

### 5.1.1 WordNet

WordNet organizza il lessico in *synset*, insiemi di lemmi sinonimi che denotano un unico *concetto*. Le relazioni collegano synset (non singole wordform) e includono: iponimia/iperonimia (*is-a*), meronimia/olonimia (*part-of*), antonimia (soprattutto per aggettivi), *see-also*, *derivationally related*, ecc. Ogni synset ha una *gloss* (definizione) e talvolta esempi d'uso.

**Iponimia/Iperonimia (*is-a*)** Sia  $x$  un synset “figlio” e  $y$  un synset “genitore”. Scriviamo  $x \sqsubseteq y$  per dire che  $x$  è **iponimo** di  $y$  (e  $y$  è **iperonimo** di  $x$ ). La relazione  $\sqsubseteq$  è un ordine parziale (riflessivo, antisimmetrico, transitivo) e induce una tassonomia (albero/DAG) per nomi e verbi.

Esempio (nomi): cane  $\sqsubseteq$  canide  $\sqsubseteq$  carnivoro  $\sqsubseteq$  mammifero  $\sqsubseteq$  vertebrato.

*Misure tassonomiche su WordNet.* Dati due synset  $x, y$  con *lowest common subsumer* (LCS)  $\text{lcs}(x, y)$  e profondità  $\text{depth}(\cdot)$  nella gerarchia, sono comuni:

$$\text{sim}_{\text{wup}}(x, y) = \frac{2 \text{depth}(\text{lcs}(x, y))}{\text{depth}(x) + \text{depth}(y)} \in (0, 1],$$

$$\text{sim}_{\text{res}}(x, y) = IC(\text{lcs}(x, y)), \quad IC(c) = -\log p(c),$$

$$\text{sim}_{\text{lin}}(x, y) = \frac{2 IC(\text{lcs}(x, y))}{IC(x) + IC(y)}$$

Qui  $p(c)$  è la probabilità empirica del concetto  $c$  stimata da corpora (conteggi gerarchici). Valori più alti indicano maggiore similarità semantica.

**Meronimia/Olonimia (*part-of*)** La **meronimia** lega un synset parte  $x$  a un synset tutto  $y$  ( $x$  *part-of*  $y$ ). In WordNet si distinguono sottotipi:

- *part meronym*: parte fisica (ruota  $\rightarrow$  auto);
- *member meronym*: membro di un collettivo (soldato  $\rightarrow$  esercito);
- *substance meronym*: sostanza costituente (cellulosa  $\rightarrow$  carta).

L'olonimia è l'inversa (*has-part*). A differenza dell'iponimia, *part-of* non definisce una tassonomia ma una gerarchia *partitiva*. È utile per inferenze “parte  $\leftrightarrow$  tutto” e per task di IE (estrazione relazioni).

#### Altre relazioni essenziali

| Relazione              | Tipo             | Esempio                             |
|------------------------|------------------|-------------------------------------|
| Antonymy               | opposizione      | caldo vs. freddo (aggettivi)        |
| Derivationally related | morfologica      | govern $\leftrightarrow$ government |
| Also-see               | semantica debole | chirurgo $\rightarrow$ medico       |
| Verb entailment        | implicazione     | snore $\Rightarrow$ sleep           |

**Lesk (gloss overlap)** Per sfruttare le *gloss*, il criterio di Lesk disambigua scegliendo il senso con massima sovrapposizione tra parole della definizione e del contesto. Sia  $G(s)$  l'insieme (o multinsieme) di lemmi nella gloss del senso  $s$  e  $C$  i lemmi di contesto: si seleziona  $\hat{s} = \arg \max_s |G(s) \cap C|$  (o versioni pesate), utile come baseline per WSD.

### 5.1.2 FrameNet

FrameNet cattura **frame** concettuali (schemi di situazioni) con i loro ruoli **Frame Elements (FE)** e le **Lexical Units (LU)** che li evocano. Un frame modella una scena prototipica (*Commercio*, *Viaggio*, *Causa\_morte*, ...).

#### Componenti principali

- **Frame.** Struttura concettuale: es. *Commercial\_transaction*.
- **Frame Elements.** Ruoli semantici (core/non-core): es. *Buyer*, *Seller*, *Goods*, *Money*, *Time*.
- **Lexical Units.** Coppie (lemma, POS) che evocano il frame: es. *buy.v*, *purchase.v*, *sell.v* (quest'ultima può evocare un frame prospettico diverso, es. *Commerce\_sell*).

L'annotazione FrameNet indica per ogni frase quali FE sono realizzati e con quali span.

#### Relazioni tra frame

| Relazione             | Intuizione                         | Esempio                                                   |
|-----------------------|------------------------------------|-----------------------------------------------------------|
| Inheritance           | specializzazione di un frame       | <i>Hiring</i> $\sqsubseteq$ <i>Commercial_transaction</i> |
| Using                 | un frame usa lo schema di un altro | <i>Buying</i> uses <i>Possession</i>                      |
| Perspective_on        | prospettive complementari          | <i>Commerce_sell</i> vs. <i>Commerce_buy</i>              |
| Subframe              | sotto-evento incluso               | <i>Travel</i> include <i>Departing</i> , <i>Arriving</i>  |
| Precedes/Causative_of | ordine causale/temporale           | <i>Paying</i> precede <i>Owning</i>                       |

### WordNet vs FrameNet

WordNet fornisce un *inventario di concetti e relazioni lessicali* (sinonimia, tassonomie e partonomie) utile per misurare similarità e per WSD. FrameNet modella *scene e ruoli* utili per SRL (Semantic Role Labeling), IE e comprensione a livello di evento. Le due risorse sono complementari: synset per “cosa è” (*is-a*) e frame per “cosa accade e chi partecipa”.

### 5.1.3 Sentiment Connotation

Per una parola  $w$  definiamo una rappresentazione connotativa di base come

$$\text{con}(w) = (\text{pol}(w), \text{int}(w)), \quad \text{pol}(w) \in [-1, 1], \text{int}(w) \in [0, 1]$$

**Polarità** ( $-1$  negativa,  $+1$  positiva) e **intensità** (forza del segno).

Se disponibile, si può estendere a coordinate affettive **VAD** (*Valence-Arousal-Dominance*); esse rappresentano la connotazione emotiva su tre assi continui: *Valence* misura il grado edonico (spiacevole  $\rightarrow$  piacevole), *Arousal* l'attivazione (calmo  $\rightarrow$  eccitato), *Dominance* il senso di controllo/potere (sottomesso  $\rightarrow$  dominante). Per una parola  $w$  definiamo un vettore  $\mathbf{v}(w) \in \mathbb{R}^3$  (normalizzato in  $[0, 1]$  o centrato in  $[-1, 1]$ ). Lo score di una frase/documento  $d = w_{1:N}$  si ottiene per media, eventualmente pesata, delle coordinate:

$$\mathbf{v}(d) = \frac{1}{Z} \sum_{i=1}^N \alpha_i \mathbf{v}(w_i), \quad \alpha_i \geq 0, \quad Z = \sum_i \alpha_i,$$

dove i pesi  $\alpha_i$  possono riflettere POS, distanza dal target o salienza sintattica. La composizione locale gestisce negazioni e intensificatori scalando le componenti (es.:  $\text{neg}(x) : v_V \leftarrow -\lambda v_V$ ;  $\text{molto}(x) : \mathbf{v} \leftarrow \kappa \mathbf{v}$  con  $0 < \lambda \leq 1$ ,  $\kappa > 1$ ).

Dato un testo  $d = w_{1:N}$  e un lessico connotativo (prior di parola), uno score medio è

$$s(d) = \frac{1}{N} \sum_{i=1}^N \text{pol}(w_i), \quad s(d) \in [-1, 1]$$

Per dare più peso ai token informativi, si introducono pesi  $\alpha_i \geq 0$  con  $\sum_i \alpha_i = 1$  (in base a POS, dipendenze, distanza dal target):

$$s_\alpha(d) = \sum_{i=1}^N \alpha_i \text{pol}(w_i).$$

Le connotazioni si *compongono* nel contesto. Su una finestra di scope (meglio: arco di dipendenza) definiamo trasformazioni semplici:

$$\begin{aligned} \text{pol}(\text{neg } x) &= -\lambda \text{pol}(x), & 0 < \lambda \leq 1, \\ \text{pol}(\text{molto } x) &= \kappa \text{pol}(x), & \kappa > 1, \\ \text{pol}(\text{poco } x) &= \mu \text{pol}(x), & 0 < \mu < 1. \end{aligned}$$

Esempi: *non buono*  $\Rightarrow$  attenuazione/inversione ( $\lambda \lesssim 1$ ); *molto utile*  $\Rightarrow$  amplificazione ( $\kappa > 1$ ); *poco convincente*  $\Rightarrow$  attenuazione ( $\mu < 1$ ). Nella pratica si usa un insieme di segnali (lista di negazioni, intensificatori, attenuatori) e si limita lo *scope* con regole sintattiche per evitare inversioni spurie.

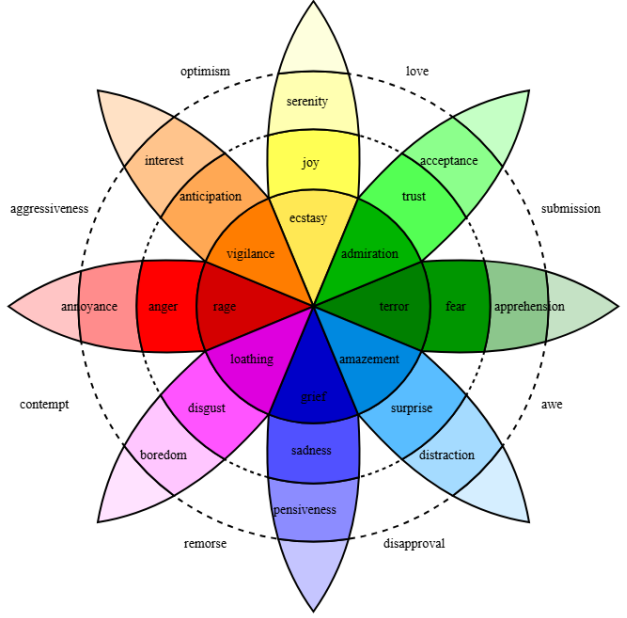


Figure 1: Plutchick's wheel of emotion

## 5.2 Vector Semantics

La *vector semantics* rappresenta il significato delle parole tramite vettori numerici in uno spazio  $\mathbb{R}^d$ . A ciascuna parola  $w$  associamo un vettore denso  $\mathbf{e}(w) \in \mathbb{R}^d$  tale che parole con significati simili risultino vicine secondo una misura di similarità (di solito la coseno):

$$\text{sim}(w_i, w_j) = \cos(\mathbf{e}(w_i), \mathbf{e}(w_j)) = \frac{\mathbf{e}(w_i) \cdot \mathbf{e}(w_j)}{\|\mathbf{e}(w_i)\| \|\mathbf{e}(w_j)\|}$$

L'idea chiave è distribuzionale: parole che compaiono in contesti simili hanno rappresentazioni simili.

Esistono due vie principali, concettualmente semplici, per ottenere vettori semantici:

- **Conteggi e matrici (count-based).** Si costruisce una matrice parola-contesto  $\mathbf{M} \in \mathbb{R}^{|V| \times |C|}$  a partire dalle co-occorrenze  $\text{count}(w, c)$  in finestre o dipendenze; si trasformano le celle con un peso informativo (es. PMI/PPMI) e, opzionalmente, si riduce la dimensionalità con SVD per ottenere vettori densi:

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}, \quad \text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0),$$

$$\mathbf{M} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top, \quad \text{vettore parola } \mathbf{e}(w) = (\mathbf{U}_k \mathbf{\Sigma}_k^p)_{w,:}, \quad p \in \{0, \frac{1}{2}, 1\}$$

Questa famiglia è trasparente: i vettori derivano direttamente dalle statistiche di co-occorrenza.

- **Predittivi (neural/predictive).** Invece di contare, si *apprendono* i vettori come parametri che rendono buone le previsioni di contesto o della parola. In forma astratta, dato un obiettivo predittivo  $\mathcal{J}(\theta)$  su coppie (parola, contesto), si ottimizzano parametri  $\theta$  (tra cui i vettori) per massimizzare la verosimiglianza o una loss contrastiva; il risultato sono embedding  $\mathbf{e}(w)$  che catturano regolarità distribuzionali. Esempi classici sono modelli che predicono  $P(w | C)$  o  $P(C | w)$  con una testa softmax e normalizzazione:

$$P(w | C) = \frac{\exp(\mathbf{e}(w) \cdot \mathbf{g}(C))}{\sum_{w' \in V} \exp(\mathbf{e}(w') \cdot \mathbf{g}(C))}$$

con  $\mathbf{g}(C)$  vettore del contesto. Gli embedding sono le righe (o colonne) delle matrici dei parametri.

Un **embedding** è una mappa  $\text{emb} : V \rightarrow \mathbb{R}^d$  che associa a ogni parola  $w \in V$  un vettore  $\mathbf{e}(w)$ . Questi vettori sono *densi*, a bassa dimensione ( $d \ll |V|$ ), e permettono di eseguire calcoli geometrici (similarità, distanza, analogie) e di fungere da input numerico per modelli supervisionati/neurali. Il principio operativo è che proprietà semantiche e sintagmatiche della parola emergono come *direzioni* e *vicinanze* nello spazio: parole con ruoli simili hanno vettori vicini, trasformazioni lineari catturano relazioni regolari, e la composizione (es. medie pesate) consente di costruire rappresentazioni per frasi brevi.



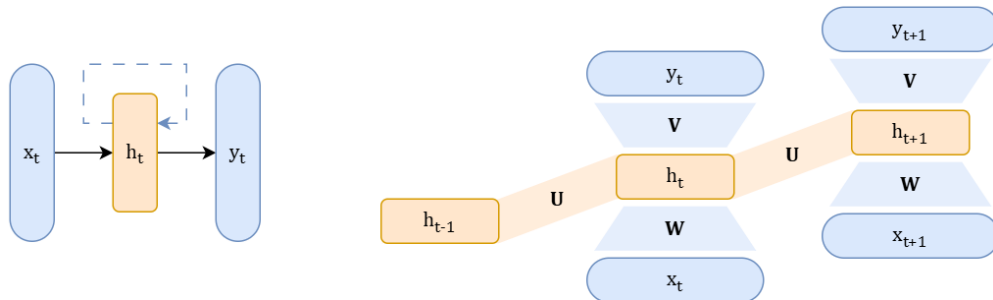


## 6 Neural Language Models

### 6.1 Recurrent Neural Networks (RNN)

Le **Reti Neurali Ricorrenti** (Recurrent Neural Networks - RNN) nascono per gestire la natura intrinsecamente sequenziale del linguaggio. A differenza dei modelli a finestra fissa (sliding windows), le RNN elaborano l'input un elemento alla volta, mantenendo una "memoria" di ciò che hanno visto in precedenza.

Il modello base, introdotto da Elman nel 1990, introduce il concetto di **stato nascosto** (hidden state) che evolve nel tempo. Al passo temporale  $t$ , la rete riceve in input il token corrente  $x_t$  e lo stato nascosto del passo precedente  $h_{t-1}$ .



Le equazioni fondamentali per l'aggiornamento dello stato e il calcolo dell'output sono:

$$h_t = g(Uh_{t-1} + Wx_t), \quad y_t = f(Vh_t)$$

Dove:

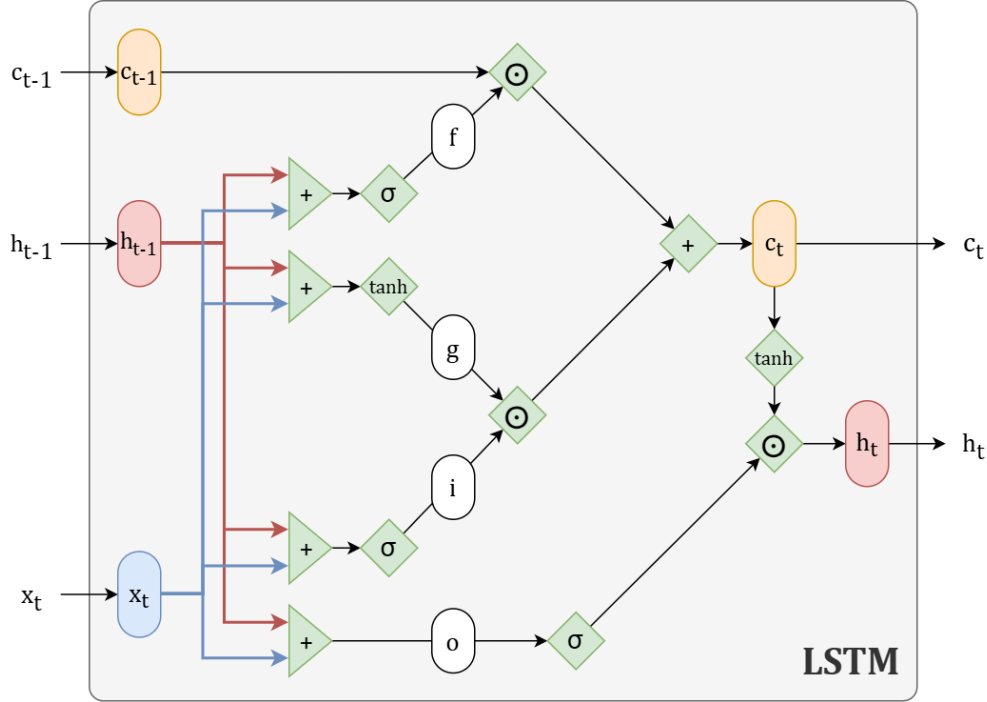
- $U, W, V$  sono le matrici dei pesi (condivise per tutti i passi temporali).
- $g$  è la funzione di attivazione (solitamente *tanh* o ReLU).
- $f$  è la funzione di output (es. *softmax* per la classificazione).

La rete può essere "srotolata" (unrolled) nel tempo, trasformandola in una lunga catena di reti feed-forward che condividono gli stessi pesi (perciò la rete non è iterata dentro se stessa, ma interconnessa ad altre reti). Questo permette di utilizzare l'algoritmo di **Backpropagation Through Time (BPTT)** per l'addestramento.

### 6.1.1 LSTM

Le RNN classiche (o "vanilla") soffrono del problema del **Vanishing Gradient** (evanescenza del gradiente). Quando si calcola l'errore e lo si propaga indietro nel tempo per molti passi, il gradiente tende a diventare piccolissimo, impedendo alla rete di imparare dipendenze a lungo; inoltre l'onere computazionali è elevato.

Per risolvere questo problema sono state introdotte le **Long Short-Term Memory (LSTM)**. Le LSTM introducono una "cella di memoria" ( $c_t$ ) separata dallo stato nascosto ( $h_t$ ) e utilizzano dei **gate** (porte) per controllare il flusso di informazioni:



- **Forget Gate ( $f_t$ )**: Decide quali informazioni dimenticare dallo stato precedente.

$$f_t = \sigma(U_f h_{t-1} + W_f x_t)$$

- **Input Gate ( $i_t$ ) e Candidate ( $g_t$ )**: Decidono quali nuove informazioni memorizzare.

$$i_t = \sigma(U_i h_{t-1} + W_i x_t), \quad g_t = \tanh(U_g h_{t-1} + W_g x_t)$$

- **Update della Cella ( $c_t$ )**: Combina la memoria vecchia (filtrata) con quella nuova.

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

- **Output Gate ( $o_t$ )**: Decide cosa esporre come output (stato nascosto).

$$o_t = \sigma(U_o h_{t-1} + W_o x_t), \quad h_t = o_t \odot \tanh(c_t)$$

### 6.1.2 Advanced Architectures

Per migliorare le capacità rappresentative delle RNN, sono state introdotte due varianti architetturali fondamentali che spesso vengono combinate tra loro.

**Stacked RNN (Reti Impilate)** In una Stacked RNN, più strati ricorrenti vengono impilati uno sopra l'altro. L'output (o la sequenza degli stati nascosti) del layer  $l$  al tempo  $t$  diventa l'input per il layer  $l + 1$  allo stesso istante temporale  $t$ . Questo permette alla rete di apprendere rappresentazioni gerarchiche più complesse: i layer inferiori tendono a catturare caratteristiche sintattiche o locali, mentre i layer superiori possono catturare concetti semantici più astratti.

**Bidirectional RNN (Reti Bidirezionali)** Le RNN standard elaborano l'input solo in avanti (dal passato al futuro). Tuttavia, in molti task l'intera frase è disponibile fin dall'inizio ed è utile conoscere anche il contesto futuro.

Le **Bi-RNN** sono composte da due reti indipendenti che operano sulla stessa sequenza:

- Una **Forward RNN** che legge la sequenza da sinistra a destra:

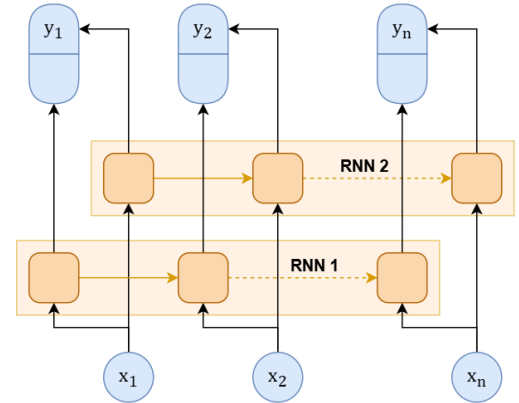
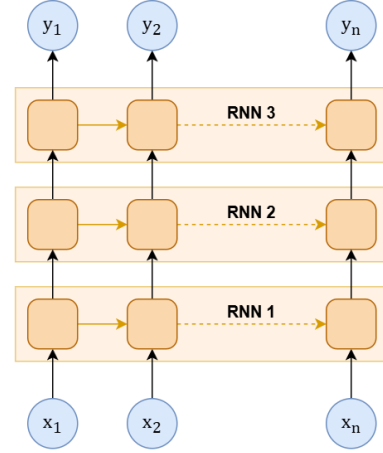
$$\vec{h}_t = RNN_{fwd}(x_1, \dots, x_t)$$

- Una **Backward RNN** che legge la sequenza da destra a sinistra:

$$\overleftarrow{h}_t = RNN_{bwd}(x_n, \dots, x_t)$$

L'output finale al passo  $t$  è solitamente la concatenazione dei due stati nascosti:

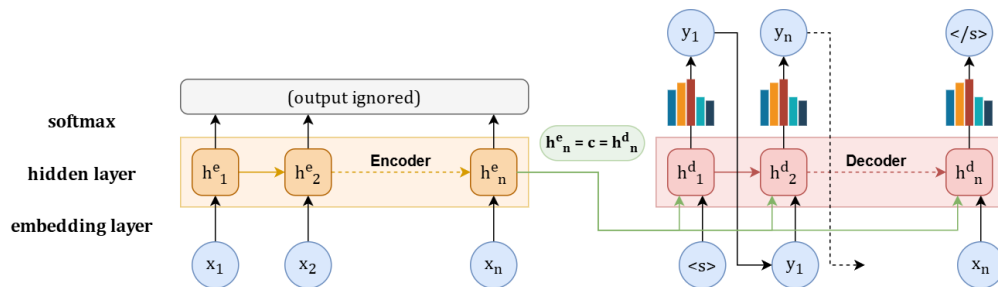
$$h_t = [\vec{h}_t \oplus \overleftarrow{h}_t]$$



### 6.1.3 Encoder-Decoder Architecture (Seq2Seq)

Per task come la traduzione automatica, dove la lunghezza dell'input differisce da quella dell'output, si utilizza l'architettura **Encoder-Decoder**:

1. **Encoder**: Una RNN legge tutta la frase di input e la comprime in un unico vettore finale chiamato *Context Vector* ( $c$ ).
2. **Decoder**: Un'altra RNN prende questo vettore  $c$  e genera la frase di output, parola per parola.



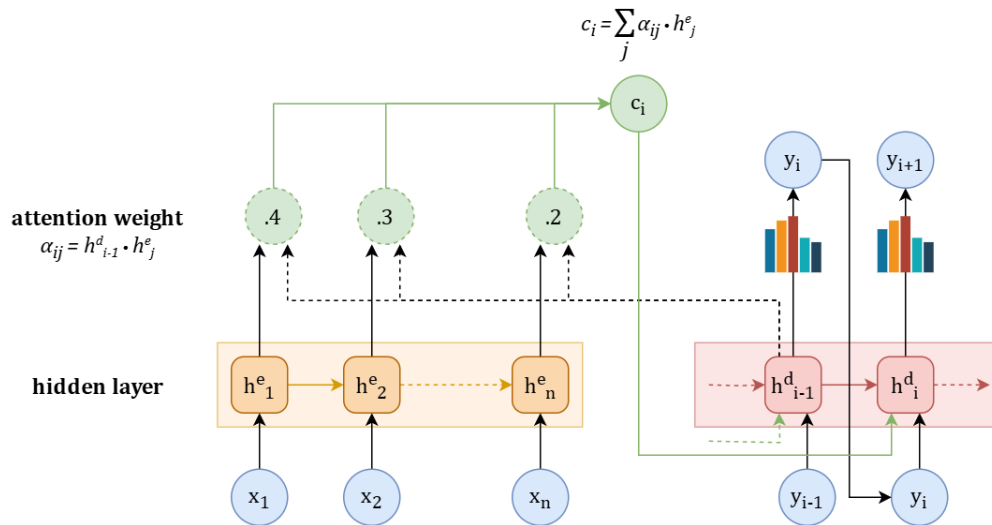
Il limite principale di questo approccio è il **Collo di Bottiglia (Bottleneck)**: costringere tutta l'informazione di una frase lunga in un unico vettore di dimensione fissa  $c$  causa perdita di informazioni.

## 6.2 Attention

Per superare il problema del "collo di bottiglia", è stato introdotto il meccanismo di **Attenzione**. L'idea chiave è che il Decoder non dovrebbe basarsi solo su un unico vettore statico, ma dovrebbe poter "guardare" (attendere) a tutti gli stati nascosti dell'Encoder a ogni passo di generazione.

### 6.2.1 Funzionamento

Ad ogni passo  $i$  del Decoder, si calcola un nuovo vettore di contesto  $c_i$  specifico per quel momento:



1. Si calcola uno **score** di similarità tra lo stato attuale del Decoder ( $h_{i-1}^d$ ) e tutti gli stati dell'Encoder ( $h_j^e$ ). Lo score più semplice è il prodotto scalare (Dot-Product):

$$score(h_{i-1}^d, h_j^e) = h_{i-1}^d \cdot h_j^e$$

Oppure con pesi addestrabili (Attention generalizzata):

$$score(h_{i-1}^d, h_j^e) = (h_{i-1}^d)^T W_s h_j^e$$

2. Si normalizzano gli score tramite una funzione **Softmax** per ottenere i pesi di attenzione  $\alpha_{ij}$  (che sommano a 1):

$$\alpha_{ij} = \frac{\exp(score(h_{i-1}^d, h_j^e))}{\sum_k \exp(score(h_{i-1}^d, h_k^e))}$$

3. Si calcola il vettore di contesto come somma pesata degli stati dell'encoder:

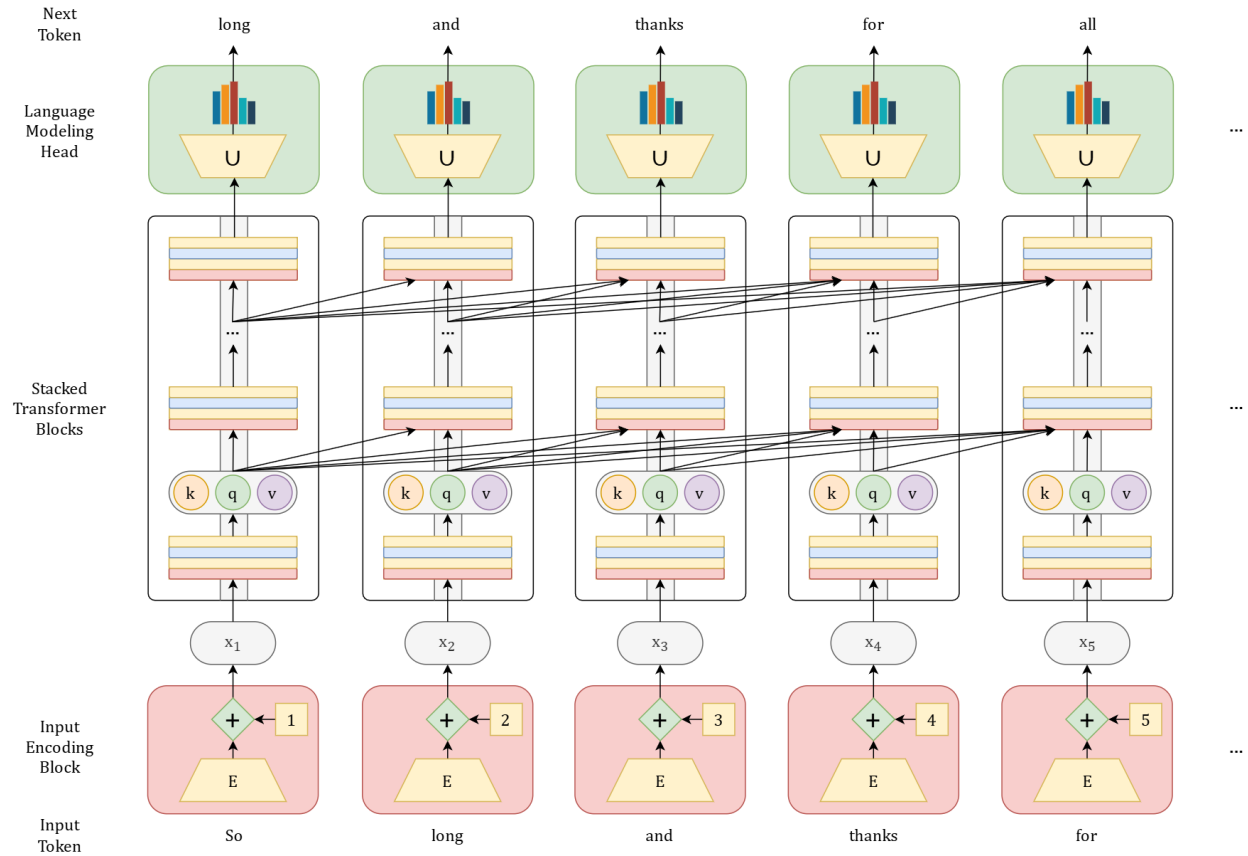
$$c_i = \sum_j \alpha_{ij} h_j^e$$

In questo modo, il modello può concentrarsi ("prestare attenzione") sulla parola rilevante della frase sorgente mentre genera la traduzione corrispondente.

### 6.3 Transformers

Nel 2017, il paper *"Attention Is All You Need"* (Vaswani et al., Google Brain) ha introdotto l'architettura Transformer, che ha rivoluzionato il NLP. A differenza delle RNN, i Transformer non processano i dati sequenzialmente ma in parallelo, basandosi interamente sul meccanismo di attenzione.

*Fare attenzione secondo certi criteri funge da strategie per selezionare i coefficienti che realmente devono guardare nella matrice.*



#### 6.3.1 Dagli Embedding Statici a quelli Contestuali

I modelli precedenti (come Word2Vec) generavano **embedding statici**: una parola aveva sempre la stessa rappresentazione vettoriale indipendentemente dal contesto. Consideriamo l'esempio:

*"The chicken didn't cross the road because **it** was too tired."*

*"The chicken didn't cross the road because **it** was too wide."*

Negli embedding statici, il vettore per "it" è identico in entrambe le frasi. Tuttavia, nel primo caso "it" si riferisce al pollo (chicken), nel secondo alla strada (road).

Il Transformer introduce gli **Embedding Contestuali**: la rappresentazione di una parola viene costruita integrando le informazioni delle parole che la circondano. Ogni parola "presta attenzione" (Self-Attention) ai vicini rilevanti per disambiguare il proprio significato.

### 6.3.2 Il Meccanismo di Self-Attention

L'attenzione è formalmente un metodo per eseguire una somma pesata di vettori. Nel Transformer, ogni token di input  $x_i$  viene proiettato in tre vettori distinti tramite matrici di pesi addestrabili ( $W^Q, W^K, W^V$ ):

- **Query** ( $q_i = x_i W^Q$ ): Rappresenta il token corrente che "cerca" informazioni.
- **Key** ( $k_i = x_i W^K$ ): Rappresenta il token come "etichetta" per essere trovato.
- **Value** ( $v_i = x_i W^V$ ): Contiene il contenuto informativo effettivo del token.

**Calcolo dello Score e dell'Output** Per calcolare quanto il token  $i$  deve prestare attenzione al token  $j$ , si calcola il prodotto scalare tra la Query di  $i$  e la Key di  $j$ . Questo valore viene scalato per la radice quadrata della dimensione delle key ( $d_k$ ) per evitare gradienti instabili:

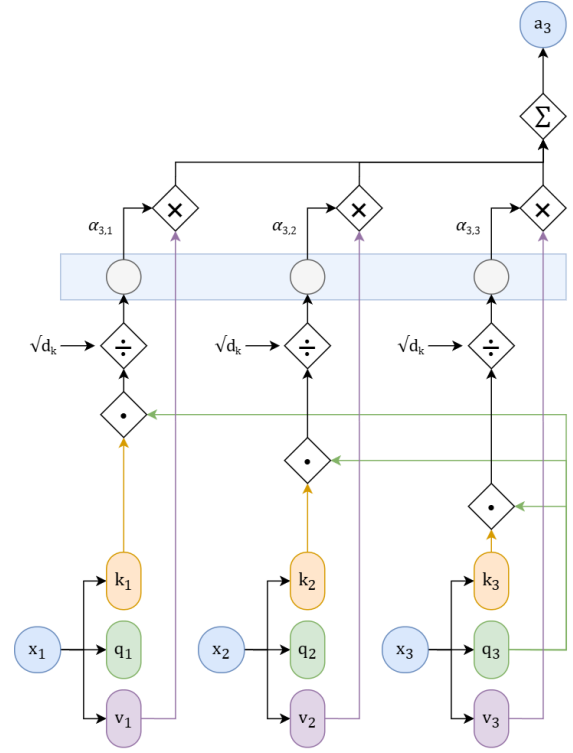
$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Successivamente si applica la Softmax per ottenere i pesi di attenzione  $\alpha_{ij}$  (che sommano a 1):

$$\alpha_{ij} = \text{softmax} \left( \frac{q_i \cdot k_j}{\sqrt{d_k}} \right)$$

L'output  $a_i$  per il token  $i$  è la somma pesata dei vettori Value degli altri token:

$$a_i = \sum_j \alpha_{ij} v_j$$



**Formulazione Matriciale** Operando su tutti i token contemporaneamente (impacchettati in una matrice  $X$ ), il calcolo diventa altamente parallelizzabile:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

### 6.3.3 Multi-Head Attention

Invece di avere una singola testa di attenzione, il Transformer ne usa molteplici ( $h$  heads). Ogni testa ha le proprie matrici indipendenti  $W_c^Q, W_c^K, W_c^V$ . L'intuizione è che diverse teste possono focalizzarsi su relazioni linguistiche diverse (es. una testa lega soggetto-verbo, un'altra aggettivo-sostantivo). Gli output delle teste vengono concatenati e proiettati linearmente tramite una matrice  $W^O$ :

$$\text{MultiHead}(X) = [\text{head}_1 \oplus \text{head}_2 \cdots \oplus \text{head}_h] W^O$$

### 6.3.4 Mascheramento (Masking)

Nel contesto del Language Modeling (predire la parola successiva), il modello non deve "vedere nel futuro". Durante il calcolo dell'attenzione  $QK^T$ , si applica una maschera che imposta a  $-\infty$  gli score relativi ai token futuri (triangolo superiore della matrice). Dopo la softmax, questi valori diventano 0, impedendo il flusso di informazioni dal futuro.

$$A = \text{softmax} \left( \text{mask} \left( \frac{QK^T}{\sqrt{d_k}} \right) \right) V$$

### 6.3.5 Il Blocco Transformer

Un blocco Transformer completo è composto da diversi sotto-strati, uniti da connessioni residuali (Residual Connections) e normalizzazione. Il flusso per un token  $x_i$  attraverso un blocco è:

1. **Multi-Head Attention:** Elabora il contesto.
2. **Add & Norm:** Si somma l'input originale all'output dell'attenzione (connessione residua) e si applica la **Layer Normalization**.

$$t^1 = \text{MHA}(x); \quad t^2 = \text{LayerNorm}(x + t^1)$$

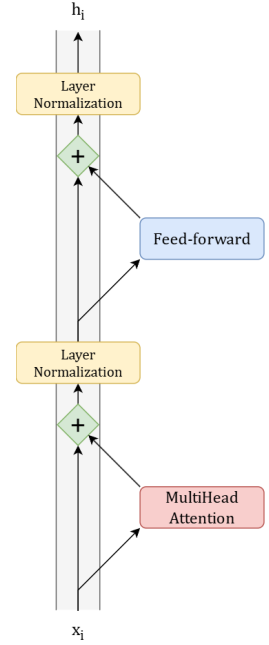
3. **Feed Forward Network (FFN):** Una rete neurale dense applicata a ogni token indipendentemente, che introduce non-linearità.

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

4. **Add & Norm:** Un'altra connessione residua seguita da normalizzazione.

$$\text{output} = \text{LayerNorm}(t^2 + \text{FFN}(t^2))$$

**Nota sul Residual Stream:** I vettori scorrono attraverso la rete lungo un "flusso residuo". I layer di attenzione leggono da questo flusso e scrivono (sommano) nuove informazioni, arricchendo progressivamente la rappresentazione del token.



### 6.3.6 Input: Token e Position Embeddings

Poiché l'attenzione è invariante all'ordine (tratta le parole come un "sacchetto"), è necessario iniettare esplicitamente l'informazione posizionale. L'input finale del Transformer è la somma di due embedding:

$$X = \text{TokenEmbedding} + \text{PositionEmbedding}$$

- **Token Embedding:** Matrice  $E$  di dimensione  $|V| \times d$ .
- **Position Embedding:** In particolare l'approccio dei *Learned Position Embeddings*, si apprende una matrice dove ogni riga rappresenta una posizione assoluta (1, 2, 3...) fino a una lunghezza massima  $N$ .

### 6.3.7 Output: La Language Modeling Head

Alla fine degli stack di blocchi Transformer, abbiamo un vettore  $h_N^L$  per l'ultimo token, che rappresenta il suo significato contestualizzato. Per predire la parola successiva:

1. Si usa un **Unembedding Layer** (spesso è la trasposta della matrice di embedding  $E^T$ , tecnica nota come *Weight Tying*). Questo proietta il vettore dalla dimensione del modello  $d$  a quella del vocabolario  $|V|$ .
2. Si ottengono i **Logits** ( $u$ ).
3. Si applica la **Softmax** per ottenere una distribuzione di probabilità su tutto il vocabolario.

$$P(w_{next}) = \text{softmax}(h_N^L E^T)$$

Da questa distribuzione si campiona il token successivo.

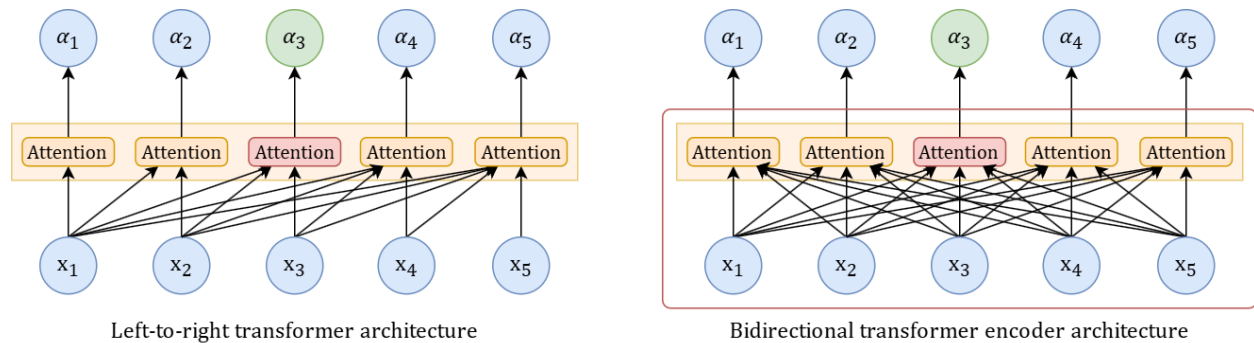




## 7 Masked Language Models and Fine Tuning

I modelli linguistici tradizionali (Causal Language Models) sono autoregressivi: predicono la parola successiva basandosi solo sul contesto di sinistra (Left-to-right transformers). Al contrario, i **Bidirectional Transformer Encoders** (come BERT) sono progettati per vedere contemporaneamente sia il contesto sinistro che quello destro.

L'architettura alla base è quella dell'Encoder del Transformer. A differenza dei modelli sequenziali classici, l'attenzione permette di mettere in relazione ogni token con tutti gli altri token della sequenza in parallelo.



### 7.1 BERT

BERT (Bidirectional Encoder Representations from Transformers) utilizza un'architettura a strati multipli di trasformatori. Le due versioni principali presentate sono:

- **BERT<sub>BASE</sub>**: 12 layer, 12 attention heads, 110M parametri.
- **BERT<sub>LARGE</sub>**: 24 layer, 16 attention heads, 304M parametri.

**Input e Tokenizzazione.** BERT gestisce la complessità computazionale limitando l'input a 512 token (con embedding posizionali fissi). La tokenizzazione utilizza l'algoritmo **WordPiece** (una variante di BPE) con un vocabolario di circa 30.000 subwords.

Rispetto al Byte Pair Encoding (BPE), l'algoritmo WordPiece addestra un semplice Language Model e, ad ogni passo, sceglie la fusione (merge) che accresce maggiormente la probabilità rispetto alla tokenizzazione (o ne minimizza la perplexity), invece di basarsi solo sulla frequenza.

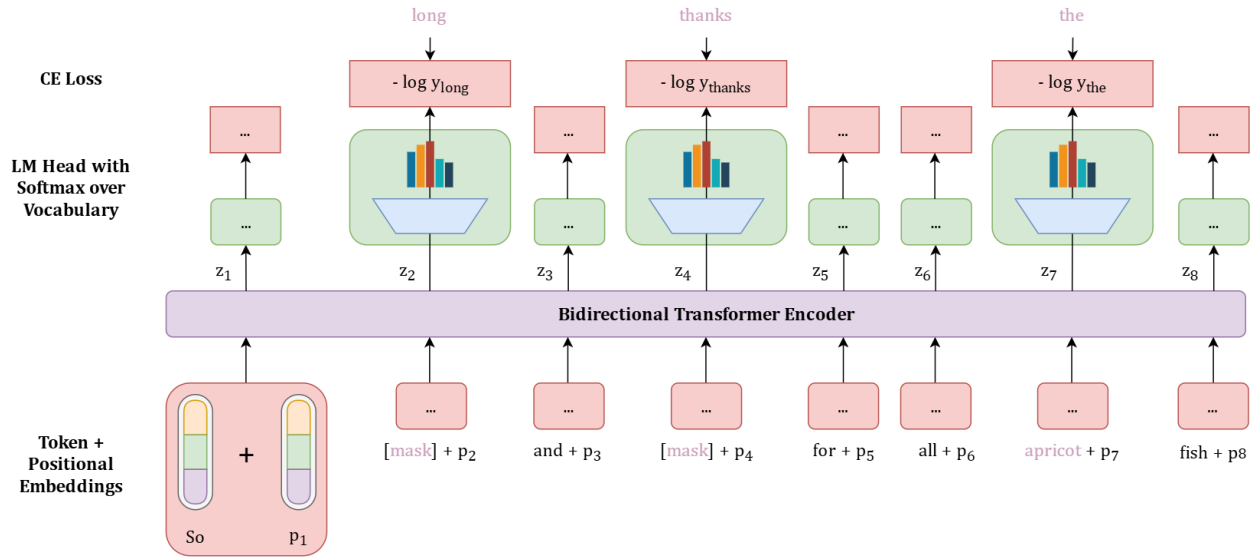
L'input finale per il modello è la somma di tre embedding distinti:

- **Token Embeddings**: la rappresentazione della subword.
- **Segment Embeddings**: per distinguere frasi diverse (es. Frase A vs Frase B).
- **Positional Embeddings**: per codificare la posizione nella sequenza.

### 7.1.1 Pre-training di BERT

Il pre-training di BERT si basa su due task non supervisionati eseguiti contemporaneamente: Masked Language Modeling (MLM) e Next Sentence Prediction (NSP).

**Task 1: Masked Language Modeling (MLM)** Poiché il modello è bidirezionale, non può essere addestrato come un normale Language Model causale (che svelerebbe la parola successiva). Si utilizza un approccio "fill-in-the-blank" (Cloze task). In generale, il mascheramento è una forma di corruzione dell'input, che può avvenire tramite sostituzioni, riordinamenti o cancellazioni.



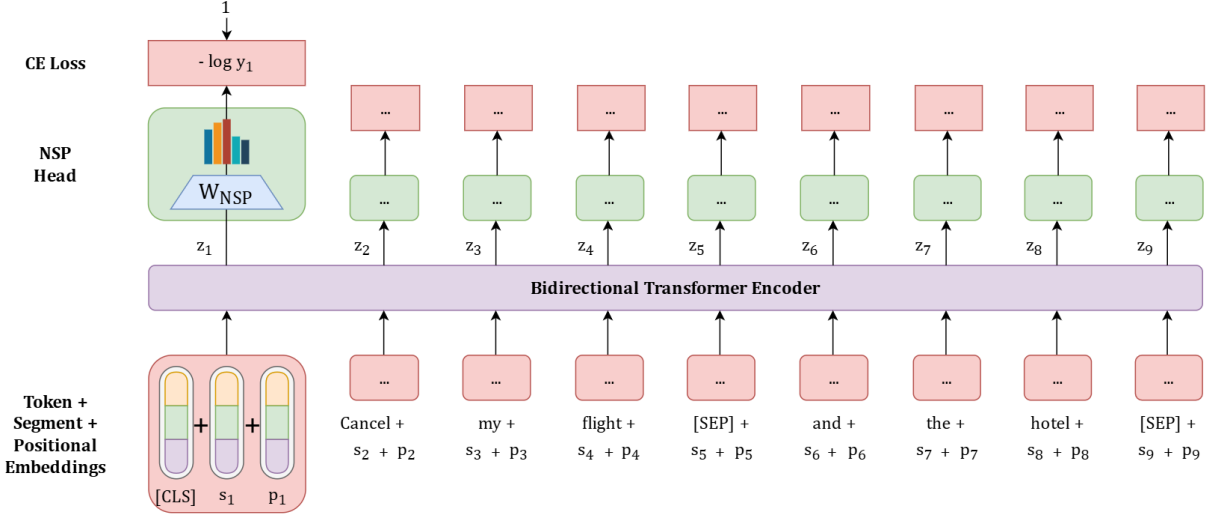
**Strategia di Masking:** Il 15% dei token di ogni sequenza viene campionato per l'apprendimento. Di questi token selezionati:

- L'80% viene sostituito con il token speciale [MASK].
- Il 10% viene sostituito con un token casuale.
- Il 10% viene lasciato invariato.

**Funzione di Costo MLM:** La perdita è calcolata tramite Cross-Entropy solo sui token mascherati. Dato  $h_i^L$  l'output dell'ultimo layer per il token  $i$ -esimo:

$$L_{MLM} = -\frac{1}{|M|} \sum_{i \in M} \log P(x_i | h_i^L)$$

**Task 2: Next Sentence Prediction (NSP)** Un altro tipo di addestramento per BERT è la Next Sentence Prediction. Molti task NLP (come Textual Entailment o QA) richiedono infatti di comprendere la relazione o la coerenza del discorso tra due frasi. L'input è formattato come: [CLS] Frase A [SEP] Frase B [SEP].



Il modello deve predire in maniera binaria se la *Frase B* segue logicamente la *Frase A*:

- 50% dei casi: B è la frase successiva reale (Positive).
- 50% dei casi: B è una frase a caso dal corpus (Negative).

La predizione avviene usando l'embedding del token speciale [CLS]. Formalmente, l'output  $y_i$  è calcolato come:

$$y_i = \text{softmax}(h_{CLS}^L W_{NSP})$$

dove  $W_{NSP} \in \mathbb{R}^{2 \times d_h}$ . La loss totale del modello durante il pre-training diviene quindi :

$$L_{TOT} = L_{NSP} + L_{MLM}$$

### 7.1.2 Embedding Contestuali

A differenza degli embedding statici (es. Word2Vec) dove ogni parola ha un vettore fisso che rappresenta il significato del *tipo* di parola, gli embedding generati da MLM sono **contestuali**; vettori che rappresentano il significato della specifica *istanza* della parola nel proprio contesto. Spesso il contextual embedding ottimale si ottiene facendo la media degli stati nascosti negli ultimi quattro layer del modello.

Questo permette di gestire la **polisemia** (es. il termine inglese "die" come verbo morire o come dado da gioco). Un'applicazione diretta è il **Word Sense Disambiguation (WSD)**. Ad esempio, tramite un algoritmo 1-nearest neighbour, si calcola l'embedding di un senso ( $v_s$ ) come media dei token contestuali ( $v_i$ ), e si assegna alla parola target  $t$  il senso che massimizza la *cosine similarity* :

$$v_s = \frac{1}{n} \sum_i v_i$$

$$\text{sense}(t) = \text{argmax}_{s \in \text{senses}(t)} \text{cosine}(t, v_s)$$

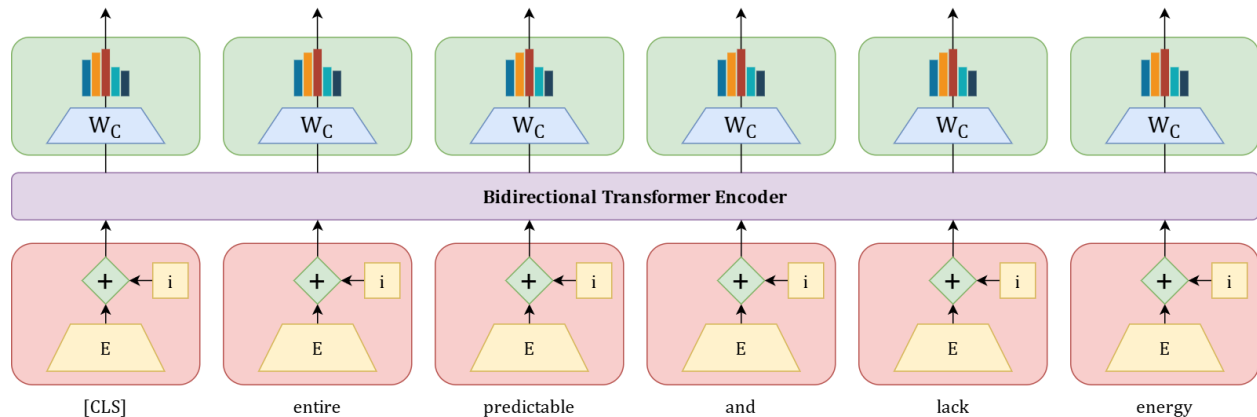
## 7.2 Fine-Tuning

Il "Transfer Learning" avviene adattando il modello pre-addestrato a task specifici aggiungendo un livello finale (head).

### 7.2.1 Classificazione di Sequenze

Si utilizza l'output del token [CLS] come rappresentazione dell'intera frase. Si aggiunge un layer denso  $W_C$ :

$$y = \text{softmax}(h_{CLS}^L W_C)$$



**Nota:** Le architetture BERT-based sono ottime per compiti di analisi di sequenze; non è detto invece che i Large Language Models (LLM) generativi arrivino alle stesse performance attuali su questi specifici task di classificazione.

### 7.2.2 Sequence Labeling (es. NER)

Per task come il Named Entity Recognition, serve una decisione per ogni token. Si applica un classificatore  $W_K$  su ogni output  $h_i^L$ . Poiché WordPiece spezza le parole, solitamente si usa la predizione del primo sub-word token per etichettare l'intera parola.

## 7.3 Varianti di BERT

- **SpanBERT** In SpanBERT si ha una nuova loss (negativa), che viene appresa a parte dalla rete e poi viene sommata alla MLM. Così facendo il modello impara a classificare interi pezzi di frasi (span) e non solo singoli token. Il processo viene effettuato per ciascuna parola mascherata all'interno dello span.
- **RoBERTa (Robustly Optimized BERT)** Sviluppato da Facebook, introduce ottimizzazioni chiave:
  - **Dynamic Masking:** la maschera cambia ogni volta che una sequenza viene passata al modello.
  - Rimuove il task NSP (Next Sentence Prediction).
  - Addestrato su una quantità di dati molto maggiore.
- **XLM e Modelli Multilingua XLM (Cross-lingual Language Model)** introduce il *Translation Language Modeling (TLM)*: l'input contiene una frase in lingua A e la traduzione in lingua B concatenate. Il modello fa attention su entrambe le lingue per predire le maschere, imparando allineamenti cross-lingua.

## 8 Large Language Models

I Large Language Models (LLM) condividono il principio di base dei modelli n-gram: assegnano probabilità a sequenze di parole e generano testo campionando la parola successiva. Tuttavia, a differenza dei modelli basati su conteggi, gli LLM sono reti neurali addestrate su enormi quantità di testo per indovinare il token successivo, apprendendo implicitamente molta conoscenza linguistica e fattuale.

### 8.1 Architetture

Esistono tre principali tipologie di architetture per gli LLM:

1. **Decoders (Decoder-only):** Noti anche come Causal o Autoregressive LLMs. Predicono le parole da sinistra a destra (es. GPT, Llama, Claude).
2. **Encoders (Encoder-only):** Come i Masked Language Models visti in precedenza (famiglia BERT). Usati principalmente per classificazione, predicono parole basandosi sul contesto sia destro che sinistro.
3. **Encoder-Decoders:** Mappano una sequenza in un'altra. Popolari per traduzione automatica e summarization (es. Flan-T5, BART).

### 8.2 Decoder-only Models e Conditional Generation

Concentriamoci sui modelli **Decoder-only**. Questi modelli eseguono una "generazione condizionata": generano testo condizionato dal testo precedente (prefix o prompt).

Molti task NLP pratici possono essere riformulati come problemi di predizione della parola successiva (Word Prediction):

- **Sentiment Analysis:** "The sentiment of the sentence 'I like Jackie Chan' is:" → il modello predice "positive".
- **Question Answering:** "Q: Who wrote The Origin of Species? A:" → il modello predice "Charles".
- **Summarization:** Si fornisce il testo originale seguito da un token speciale come "`<tl;dr>`" e il modello genera il riassunto.

| Original text                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Summarized text                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For \$89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box - enough for 10 to 15 snowballs, he says.</p> <p>But not if you live in New England or surrounding states. "We will not ship snow to any states in the northeast!" says Waring's website, ShipSnow Yo.com. "We're in the business of expunging snow!"</p> <p>His website and social media accounts claim to have filled more than 133 orders for snow - more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.</p> <p>According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. [...]</p> | <p>Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box - enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.</p> |

### 8.3 Decoding e Strategie di Campionamento

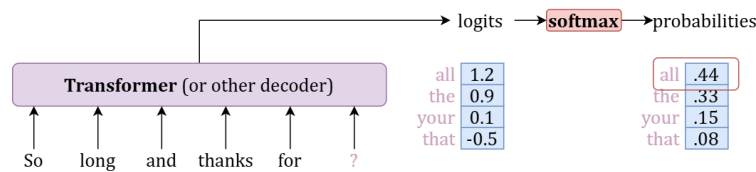
Il *decoding* è il processo di scelta della parola da generare basandosi su un criterio applicato alle probabilità fornite in output dal modello. I layer neurali degli LLM generano un vettore di logit  $u$ , che viene trasformato in una distribuzione di probabilità  $y$  tramite la funzione softmax. Le strategie di decodifica si dividono in due grandi famiglie: **deterministiche** e **stocastiche**.

#### 8.3.1 Decoding Deterministico

Queste strategie effettuano scelte prevedibili massimizzando le probabilità, ma i testi generati tendono a risultare ripetitivi e poco interessanti.

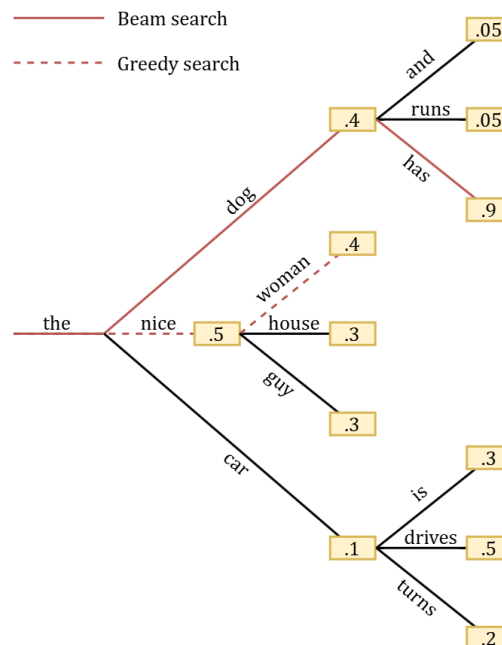
**Greedy Search** È il metodo di decodifica più semplice: fornisce ad ogni passo la scelta localmente migliore, selezionando la parola con la probabilità più alta. Formalmente, il token generato è:

$$w_t = \operatorname{argmax}_w P(w|w_{1:t-1})$$



Il limite principale di questo algoritmo è che può produrre frasi ripetitive e non sempre garantisce il risultato globale migliore, poiché potrebbe scartare percorsi ottimali nascosti dietro un token con probabilità localmente bassa.

**Beam Search** Questo algoritmo riduce il rischio di perdere sequenze di parole con probabilità globalmente elevata mantenendo in memoria un numero fisso di ipotesi candidati, definito dal parametro *num\_beam*. Ad ogni passo, esplora le continuazioni delle ipotesi correnti e seleziona le *num\_beam* sequenze con la probabilità cumulativa maggiore. Quando *num\_beam* = 1, l'algoritmo coincide esattamente con la greedy search.



### 8.3.2 Decoding Stocastico (Sampling)

Per superare la ripetitività dei metodi deterministici si introduce un elemento randomico, campionando i token da una distribuzione. Questa famiglia di metodi richiede di bilanciare un compromesso: favorire i token più probabili aumenta la qualità (testi accurati, coerenti e fattuali), mentre esplorare token a media probabilità migliora la diversità e creatività, a rischio però di minore coerenza (allucinazioni).

**Random (Multinomial) Sampling** Il token successivo viene selezionato in modo casuale, ma in misura proporzionale alla sua probabilità calcolata dal modello rispetto alle scelte precedenti:  $w_i \sim p(w_i|w_{<i})$ . Poiché anche le parole strane o fuori contesto mantengono una piccola probabilità, sommate costituiscono una fetta rilevante della distribuzione (la "coda lunga") che, se campionata, porta a frasi senza senso.

**Top-k Sampling** Per escludere la coda lunga, questa tecnica imposta un limite  $k$  fisso. Il modello calcola la *likelihood* di ogni parola nel vocabolario e mantiene solo le  $k$  parole più probabili. I punteggi di queste parole vengono poi rinormalizzati in una distribuzione di probabilità valida dalla quale si campiona casualmente. Il limite di questa strategia è la rigidità: avendo  $k$  fisso, copre quantità molto diverse di massa di probabilità a seconda che la distribuzione del modello in un certo contesto sia piatta o molto piccata.

**Top-p (Nucleus) Sampling** Invece di un numero fisso di candidati, si mantiene fissa la massa di probabilità totale per implementare una selezione adattiva (un valore di  $k$  dinamico). Dato il contesto, il vocabolario candidato  $V^{(p)}$  è l'insieme più piccolo di parole la cui somma delle probabilità raggiunge una determinata soglia  $p$ :

$$\sum_{w \in V^{(p)}} P(w|w_{<t}) \geq p$$

**Temperature Sampling** Questa tecnica non tronca l'insieme dei candidati, ma ne rimodella la distribuzione sfruttando l'iperparametro di temperatura  $\tau$ . Il vettore dei logit  $u$  viene diviso per  $\tau$  prima di essere passato alla funzione softmax:  $y = \text{softmax}(u/\tau)$ .

- $\tau < 1$  (*Low temperature*): Spinge verso 1 i valori alti e verso 0 i valori bassi, enfatizzando i token più probabili e comportandosi in maniera più "greedy".
- $\tau \rightarrow 1$ : La distribuzione originale rimane sostanzialmente invariata.
- $\tau > 1$  (*High temperature*): Il sistema diventa più "caldo" e flessibile, la distribuzione si appiattisce tendendo a quella uniforme, esplorando più combinazioni possibili.

**Contrastive Search** Questa strategia avanzata mira a generare token che siano al contempo probabili per il modello e discriminativi rispetto al contesto precedente. Il token  $x_t$  è selezionato dai top- $k$  candidati bilanciando la probabilità intrinseca (*model confidence*) con una penalità di degenerazione che sfrutta la *cosine similarity* ( $s$ ) degli embedding:

$$x_t = \arg \max_{v \in V^{(k)}} \left\{ (1 - \alpha) \times \frac{p_\theta(v|x_{<t})}{\text{model confidence}} - \alpha \times (\max\{s(h_v, h_{x_j}) : 1 \leq j \leq t-1\}) \right\}$$

L'iperparametro  $\alpha$  regola il peso di queste componenti: aumentando  $\alpha$ , il token si discosta maggiormente dal contesto precedente; se  $\alpha = 0$ , la ricerca degenera nella semplice greedy search.



## 8.4 Pre-training

L'algoritmo di training è **Self-supervised**: l'etichetta (label) è semplicemente la parola successiva nel testo. Si utilizza la **Cross-Entropy Loss**:

$$L_{CE} = - \sum_{w \in V} (y_t[w]) \log(\hat{y}_t[w]) = - \log \hat{y}_t[w_{t+1}]$$

Dove  $w_{t+1}$  è la parola corretta successiva.

Durante il training si usa il **Teacher Forcing**: al passo  $t + 1$ , il modello riceve in input la parola corretta  $w_{t+1}$  (ground truth) e non quella che ha predetto al passo precedente.

### 8.4.1 Dati di Pre-training

I modelli sono addestrati su enormi porzioni del web (es. Common Crawl, C4, The Pile). È necessario filtrare i dati per qualità (rimuovere boilerplate, deduplicazione) e sicurezza (tossicità, PII - Personally Identifiable Information). Rimangono aperti problemi legali legati al copyright e al "Fair Use".

## 8.5 Fine-Tuning dei LLM

Il *fine-tuning* è il processo mediante il quale un modello pre-addestrato (Foundation Model) viene sottoposto ad ulteriori iterazioni di addestramento su nuovi dati specifici, al fine di adattarlo a un particolare dominio (es. medico, legale, finanziario) o a task specifici.

Esistono diverse tipologie e strategie per effettuare il fine-tuning di un LLM:

- Aggiunta di una **Head** (classificatore) in cima al modello.
- **Continued Pretraining**: si continua ad addestrare tutto il modello su nuovi dati con la stessa loss.
- **Supervised Finetuning (SFT)**: per instruction tuning (dataset di prompt-risposta).

### 8.5.1 Parameter-Efficient Fine Tuning (PEFT)

Riapplicare il training a tutti i parametri del modello (*full fine-tuning*) è un'operazione estremamente lenta e costosa dal punto di vista computazionale. Le tecniche PEFT superano questo ostacolo congelando la maggior parte dei pesi pre-addestrati e aggiornando solo un piccolo sottoinsieme di parametri (spesso solo negli ultimi layer) in modo efficiente.

## 8.6 Valutazione dei LLM

Le prestazioni dei Large Language Models vengono misurate secondo diversi criteri:

- **Accuracy e Metriche Task-Oriented**: Come F1-Score, BLEU, ROUGE.
- **Costi e Prestazioni**: Velocità di esecuzione ed efficienza energetica (es. kWh consumati, emissioni  $CO_2$ ).
- **Fairness (Equità)**: Valutazione della presenza di bias interni o stereotipi di genere/razziali generati.

### 8.6.1 Log-Likelihood e Perplexity

Poiché i modelli generano testo in base alla probabilità congiunta, potremmo confrontare i modelli usando la *log-likelihood*. Tuttavia, essa diminuisce all'aumentare della lunghezza del testo.

La metrica per valutare i LM è la **Perplexity** (inverso della probabilità normalizzata per la lunghezza):

$$PP(W) = \sqrt[n]{\prod_{i=1}^n \frac{1}{P(w_i|w_{<i})}}$$

Minimizzare la perplessità equivale a massimizzare la probabilità del test set.

È importante notare che la perplexity è sensibile al processo di tokenizzazione; per confrontare due modelli diversi è necessario che utilizzino lo stesso tokenizer.

## 8.7 Scalabilità e Tecniche di Ottimizzazione

La scalabilità dei modelli è studiata per comprendere come migliorare le performance e come gestire LLM con centinaia di miliardi di parametri (come Llama 3.1 405B) in scenari con risorse limitate.

## 8.8 Leggi di Scaling (Scaling Laws)

Le prestazioni di un LLM (misurate tramite la loss  $L$ ) scalano secondo una legge di potenza rispetto a tre fattori principali (mantenendo costanti gli altri):

1. Dimensione del modello ( $N$ ):  $L(N) = (\frac{N_c}{N})^{\alpha_N}$
2. Dimensione del dataset ( $D$ ):  $L(D) = (\frac{D_c}{D})^{\alpha_D}$
3. Budget di computazione ( $C$ ):  $L(C) = (\frac{C_c}{C})^{\alpha_C}$

Queste equazioni empiriche permettono di stimare la loss finale già all'inizio del training.

Il numero di parametri non-embedding  $N$  può essere approssimato come:  $N \approx 12 \cdot n_{layer} \cdot d^2$  (assumendo  $d_{attn} = d_{ff}/4 = d$ ).

### 8.8.1 KV Cache

Mentre durante il training la self-attention viene calcolata efficientemente in parallelo tramite  $A = \text{softmax}(\frac{QK^\top}{\sqrt{d_k}})V$ , in fase di inferenza la generazione è autoregressiva (un token alla volta).

Per evitare di ricalcolare le matrici Key ( $K$ ) e Value ( $V$ ) per tutti i token generati in precedenza  $X_{<j}$ , questi vettori vengono memorizzati in una struttura chiamata **KV Cache**, permettendo al modello di recuperarli direttamente per i passaggi successivi, riducendo drasticamente il costo computazionale.

### 8.8.2 LoRA (Low-Rank Adaptation)

LoRA è una popolare tecnica PEFT usata per ottimizzare il fine-tuning. Nei layer di self-attention ci sono grandi matrici di pesi densi ( $W \in \mathbb{R}^{d \times d}$ ). Invece di aggiornare l'intera matrice ( $W \rightarrow W + \Delta W$ ), LoRA congela i pesi originali e apprende solo un aggiornamento approssimato a basso rango (*Low-Rank*) tramite la decomposizione in due matrici  $A$  e  $B$ :

- $A \in \mathbb{R}^{N \times r}$
- $B \in \mathbb{R}^{r \times d}$

con il rango  $r \ll \min(N, d)$ . Durante il *forward pass*, l'operazione diventa quindi:  $h = xW + xAB$ . Il numero di parametri aggiornabili diventa minuscolo, abbattendo drasticamente l'uso di memoria VRAM.

### 8.8.3 Quantizzazione

I parametri dei modelli sono convenzionalmente salvati nel formato floating point a 32-bit (FP32). La quantizzazione è una tecnica di compressione che riduce la precisione con cui i pesi vengono salvati (ad esempio a FP16, FP8 o interi INT8), in modo da ridurre notevolmente lo spazio occupato in memoria e accelerare l'inferenza, con una perdita marginale in termini di performance.



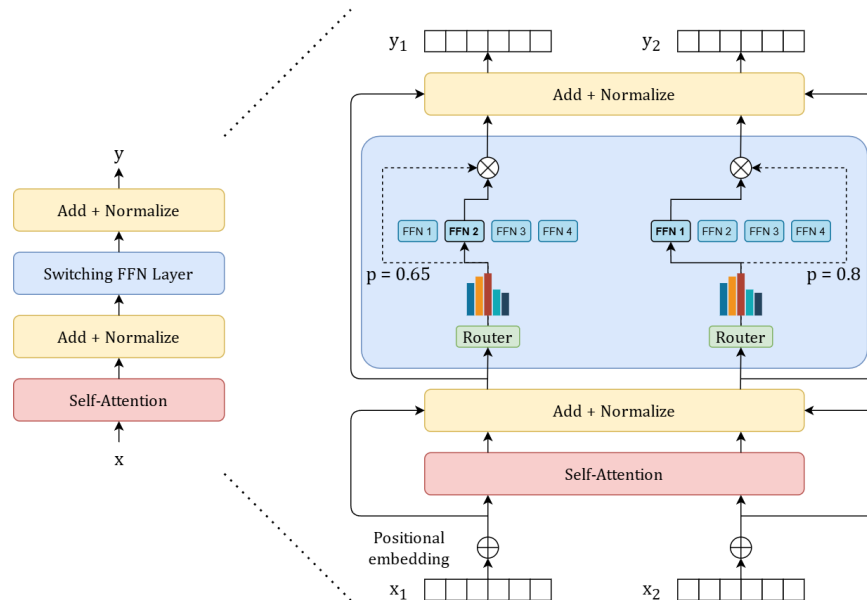
## 9 Common LLM Architectures

I modelli linguistici di grandi dimensioni (LLM) possono essere classificati in base al loro scopo e alla loro architettura:

- **Embedding LLM e Generator LLM:** I primi, tipicamente *Encoder-only*, generano rappresentazioni vettoriali del testo in input. I secondi, di tipo *Decoder-only*, generano testo in linguaggio naturale a partire da un prompt.
- **Foundational e Instruction-tuned:** I modelli fondazionali (o di base) sono general-purpose e vengono addestrati su enormi quantità di dati con tecniche *self-supervised* per catturare la conoscenza linguistica. I modelli instruction-tuned vengono invece addestrati su dataset annotati, tramite tecniche supervisionate (Supervised Fine Tuning - SFT), per seguire specifiche istruzioni fornite in input.
- **Multilingual e Multimodal:** Esistono modelli capaci di operare su più lingue e modelli multimodali che possono ricevere in input non solo testo, ma anche immagini, video e audio.

### 9.1 Architettura Mixture of Experts (MoE)

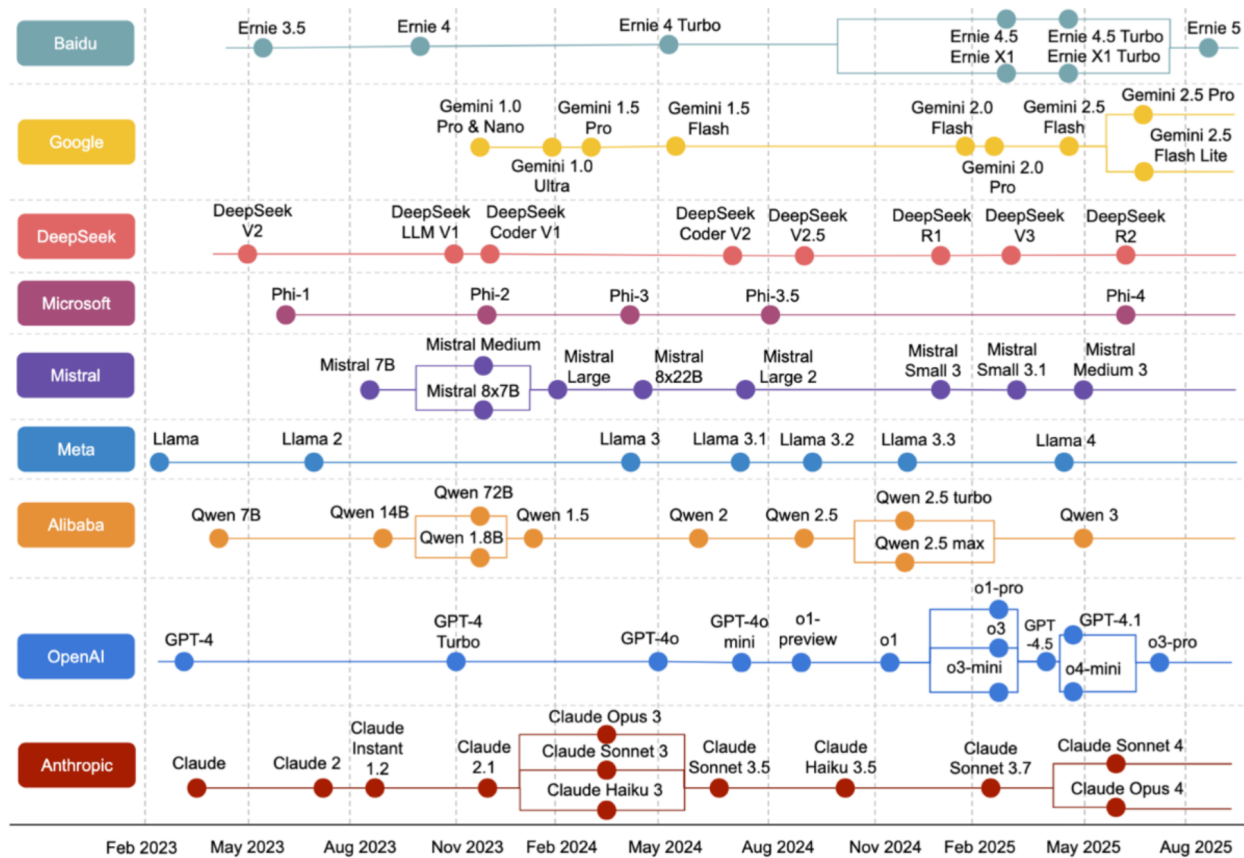
L'aumento della dimensione di un modello garantisce in genere prestazioni migliori, ma richiede un budget di calcolo elevato. Un'architettura *Mixture of Experts* (MoE) rappresenta un compromesso: permette di pre-addestrare modelli aumentandone drasticamente i parametri senza incrementare il budget di calcolo rispetto a un modello denso.



Un modello MoE si compone di due elementi chiave:

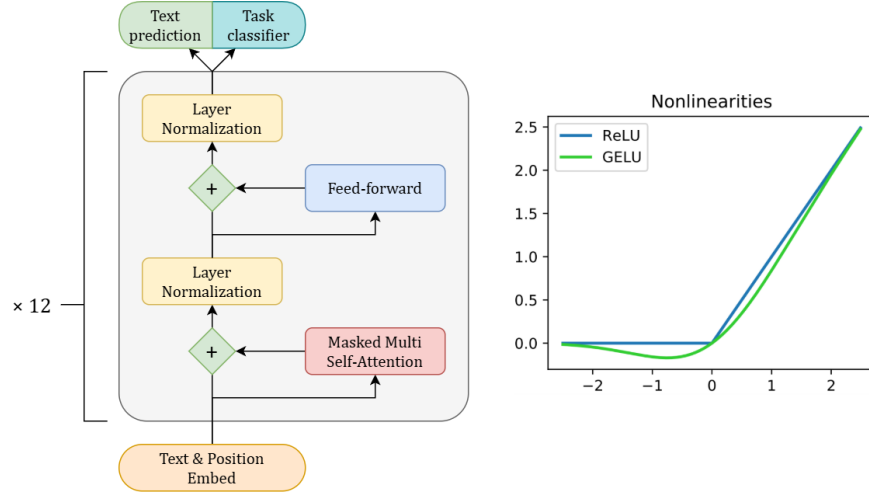
1. **Esperti (Layer MoE sparsi):** Sostituiscono i tradizionali layer densi Feed-Forward Network (FFN). Ogni esperto è una rete neurale specializzata (su specifici gruppi di token o concetti, come la punteggiatura o i nomi propri).
2. **Gate Network (Router):** Determina a quali esperti inviare ciascun token. Il router ha parametri apprendibili e viene pre-addestrato insieme al resto della rete.

Nonostante i MoE siano molto efficienti nel pre-training e consentano un'inferenza più rapida rispetto ai modelli densi (poiché solo una frazione dei parametri viene attivata), storicamente soffrono di overfitting durante il fine-tuning e presentano requisiti di memoria RAM elevati, dato che tutti i parametri devono essere caricati.



## 9.2 La Famiglia GPT

La famiglia Generative Pre-trained Transformer (GPT), sviluppata da OpenAI, si basa su architetture *decoder-only* sottoposte a pre-training per il Language Modeling e successivo fine-tuning supervisionato. Il modello GPT originale utilizza uno stack di 12 blocchi transformer decoder e adotta la funzione di attivazione GeLU al termine di ogni blocco FFN.



La funzione GeLU (Gaussian Error Linear Unit) modula l'attivazione lineare tramite la distribuzione cumulativa gaussiana ed è approssimata dalla seguente formula:  $GELU(x) \sim x\sigma(1.702x)$

### 9.2.1 Pre-training e Fine-tuning in GPT

Durante il pre-training non supervisionato, dato un insieme di token  $U = \{u_1, \dots, u_n\}$ , il modello calcola la probabilità attraverso le seguenti trasformazioni, partendo dagli embedding testuali e posizionali:

- $h_0 = UW_e + W_p$
- $h_i = \text{transformer\_block}(h_{i-1}) \forall i \in [1, n]$
- $P(u) = \text{softmax}(h_n W_e^T)$

La funzione di loss per il pre-training massimizza la predizione del token successivo:

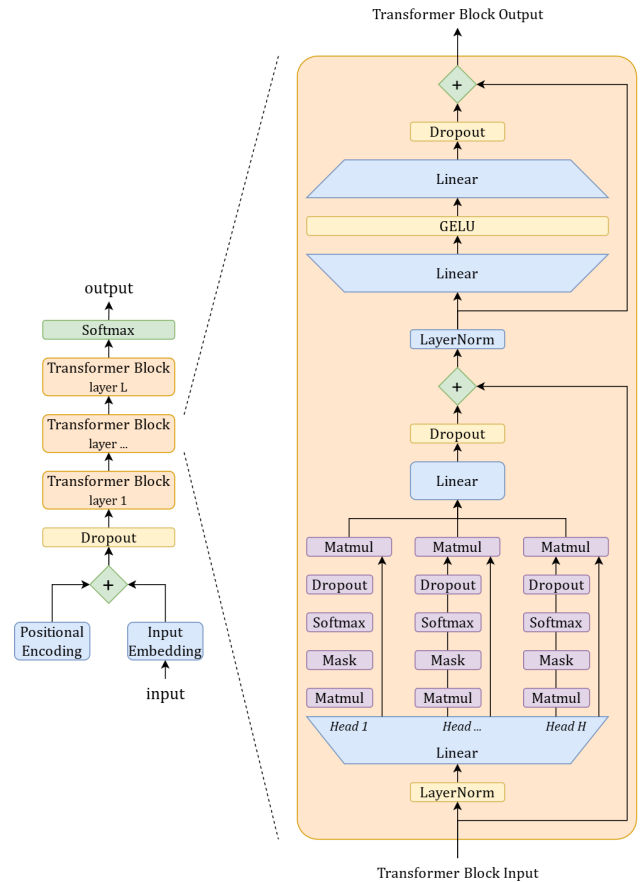
$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

Nel fine-tuning supervisionato, viene aggiunto un layer lineare finale per predire un'etichetta  $y$  da una sequenza  $x^1, \dots, x^m$ :

$$P(y | x^1, \dots, x^m) = \text{softmax}(h_l^m W_y)$$

La funzione obiettivo finale combina la loss del pre-training e quella del fine-tuning:

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$



### 9.3 Architetture Ottimizzate: LLaMA e Mistral

Modelli più recenti hanno introdotto varianti architetturali per migliorare stabilità e prestazioni.

#### 9.3.1 LLaMA (Meta)

La serie LLaMA utilizza una versione modificata dell'architettura transformer decoder-only con alcune differenze chiave rispetto a GPT:

- **SwiGLU**: Sostituisce la ReLU. Combina le funzioni Swish e Gated Linear Unit. La formulazione integra le attivazioni:

$$\text{SwiGLU}(x) = x \cdot \sigma(\beta x) + (1 - \sigma(\beta x)) \cdot \sigma(W \cdot x + b)$$

- **Rotary Positional Embedding (ROPE)**: Codifica la posizione relativa tra i vettori di self-attention direttamente attraverso matrici di rotazione. Per i vettori query e key, la relazione diventa:

$$q_m^\top k_n = x_m^\top W_q^\top R_{\theta, n-m} W_k x_n$$

- **RMSNorm**: Sostituisce la LayerNorm classica per ridurre il carico computazionale e stabilizzare il training. La formula è:

$$\hat{x} = \frac{x}{\text{RMS}(x) + \epsilon}$$

#### 9.3.2 Mistral

I modelli Mistral introducono tecniche avanzate di gestione dell'attention per migliorare il throughput e la velocità:

- **Grouped-Query Attention (GQA)**: Le *head* delle query sono divise in gruppi, e ciascun gruppo condivide una singola *head* per chiavi (keys) e valori (values), riducendo la memoria richiesta durante il decoding.
- **Sliding Window Attention (SWA)**: Consente di gestire sequenze di contesto più lunghe riducendo i costi computazionali attraverso finestre di attenzione fisse.

### 9.4 Altri Modelli e Approcci

- **Claude (Anthropic)**: Focus sull'allineamento tramite *Constitutional AI* e RLHF, per garantire risposte etiche e sicure basate su principi guida predefiniti.
- **Gemini e Gemma (Google)**: Gemini sfrutta MoE multimodali e Multi-Query Attention (MQA). Gemma è una versione open-source "distillata" (Knowledge Distillation) da Gemini, disponibile in versioni pre-addestrate (PT) e instruction-tuned (IT).
- **DeepSeek**: Implementa la *Multi-head Latent Attention (MLA)*, che utilizza un'approssimazione a basso rango per comprimere lo spazio della cache Key-Value (KV), risultando estremamente efficiente in fase di inferenza.

## 10 Prompting

Il *prompt* è l'istruzione fornita in input a un Large Language Model (LLM). La stringa immessa viene elaborata dal modello, che genera iterativamente nuovi token condizionati dal contesto creato dal prompt stesso, al fine di raggiungere l'obiettivo dell'utente. Il processo di ricerca e ottimizzazione di prompt efficaci per specifici task è noto come *Prompt Engineering*.

Per evitare di scrivere un nuovo prompt per ogni singola interrogazione, si utilizzano i *Template*. Essi garantiscono scalabilità e devono essere progettati in modo non ambiguo. Il flusso di utilizzo prevede:

1. Sviluppo di un template specifico per il task con un parametro libero per l'input testuale.
2. Istanziamento del prompt compilato inserendo l'input nel template.
3. Decodifica autoregressiva da parte del LLM per generare i token di output.
4. Estrazione della risposta desiderata dall'output del modello.

**Few-shot Prompting** Per migliorare le prestazioni del modello, è possibile includere nel prompt alcuni esempi etichettati del task (dimostrazioni), tecnica nota come *few-shot prompting*. Al contrario, lo *zero-shot prompting* non prevede alcun esempio. Gli esempi devono essere pochi; il maggior guadagno in termini di performance si ottiene solitamente dal primo esempio inserito, mentre per i successivi l'incremento diminuisce. Gli esempi possono essere selezionati:

- **Automaticamente:** selezionando nel training set gli esempi con gli embedding più simili a quello corrente.
- **Programmaticamente:** sfruttando una metrica di performance del task per massimizzare il risultato.

### 10.1 In-Context Learning e Induction Head

La capacità di un LLM generativo di apprendere dal contesto fornito nel prompt e migliorare la previsione dei token è definita *in-context learning*.

Il funzionamento di questo meccanismo si basa sull'ipotesi delle *induction head*. Si tratta di particolari *attention head* che riconoscono pattern ripetitivi del tipo  $\{A, B, \dots, A\} \rightarrow B$ . Il meccanismo prevede due fasi:

- **Previous token head:** copia il token precedente nel *residual stream* del token attuale.
- **Match-and-copy:** l'induction head cerca una corrispondenza tra una *query* derivata dal token corrente e una *key* generata dall'output della *token head* precedente, spostando il valore (Value) nell'output.

### 10.2 Model Alignment

A causa della loro natura autoregressiva, i modelli pre-addestrati possono ignorare il condizionamento del prompt e generare testi dannosi, tossici o falsi. Il *Model Alignment* comprende i metodi atti ad adattare l'LLM alle esigenze umane. Esistono due strategie principali: *Instruction Tuning* e *Preference Alignment*.

#### 10.2.1 Instruction Tuning (SFT)

L'Instruction Tuning (o Supervised Fine Tuning - SFT) consiste nell'effettuare un fine-tuning del modello pre-addestrato su un corpus di istruzioni e relative risposte. Questo approccio attiva una forma di *meta-learning*: il modello non impara solo i singoli task, ma migliora globalmente la sua capacità di seguire istruzioni.

I dati per il training possono provenire da dataset curati manualmente (molto dispendiosi) oppure possono essere generati in automatico trasformando task classici tramite template o chiedendo a un LLM di generare e parafrasare istruzioni partendo da linee guida umane. La valutazione di questi modelli avviene solitamente tramite un approccio *Leave-one-out*, rimuovendo un intero cluster di task dal corpus di addestramento per usarlo solo in fase di test.



## 10.3 Tecniche Avanzate di Prompting

### 10.3.1 Chain-of-Thought (CoT)

Il *Chain-of-Thought prompting* migliora le prestazioni su task di ragionamento complessi. L'idea è indurre il modello a suddividere il problema in passaggi logici intermedi (step by step), proprio come farebbe un essere umano. Nel few-shot prompting con CoT, le dimostrazioni includono esplicitamente il testo che spiega il ragionamento.

### 10.3.2 Automatic Prompt Optimization (APO)

L'APO è una tecnica che cerca automaticamente i prompt più performanti eseguendo una ricerca iterativa nello spazio dei prompt. Un algoritmo di ricerca necessita di:

1. **Stato iniziale:** prompt di partenza.
2. **Metrica di scoring:** valutazione delle performance mescolando prompt e campioni del training set (spesso usando algoritmi come la *Beam Search*).
3. **Metodo di espansione:** generazione di varianti del prompt, solitamente chiedendo a un LLM di effettuare una parafrasi.

Esiste anche la *Informed Search*, in cui si chiede a un LLM di criticare un prompt sulla base degli esempi in cui ha fallito, generando poi una versione migliorata.

## 10.4 Preference Alignment: RLHF e DPO

### 10.4.1 Reinforcement Learning with Human Feedback (RLHF)

L'RLHF sfrutta il feedback umano per addestrare un modello di reward (*Reward Model*), basato su come gli addestratori umani classificano le risposte dell'LLM. Questo reward model guida poi un algoritmo di reinforcement learning per ottimizzare i pesi del modello principale.

Il motore di ottimizzazione tipico è il **Proximal Policy Optimization (PPO)**. PPO usa la *gradient ascent* con una funzione di clipping per evitare che la nuova policy diverga eccessivamente dalla precedente, garantendo stabilità e controllo. È ideale per task complessi che richiedono pianificazione a lungo termine e sviluppo su larga scala.

### 10.4.2 Direct Preference Optimization (DPO)

Il DPO è un approccio alternativo che ottimizza direttamente i parametri del modello sulle preferenze umane, bypassando la creazione del modello di reward e riducendo la complessità di calcolo e di tuning degli iperparametri. Funziona raccogliendo dataset con rating o comparazioni binarie, sviluppando una *loss function* che calcola la discrepanza tra gli output generati e le preferenze umane e aggiornando i parametri.

Il DPO risulta essere più leggero e adattabile rapidamente ai feedback, adatto per task semplici o contesti con risorse limitate.

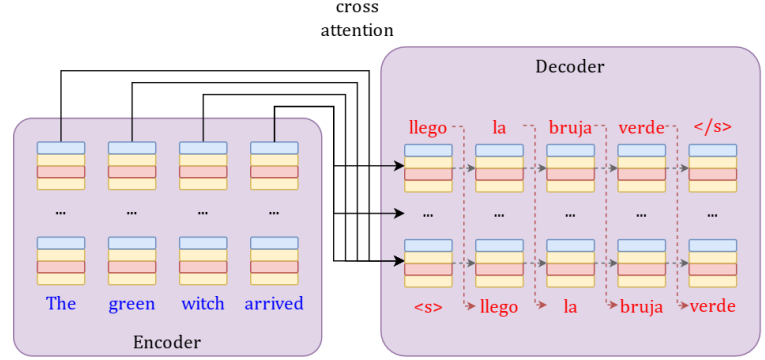
## 11 Generative Tasks & Evaluation

### 11.1 Task Generativi

I modelli linguistici possono essere utilizzati per affrontare diversi task generativi, tra cui la traduzione automatica, la classificazione e il question answering.

#### 11.1.1 Machine Translation (MT)

Il task di Machine Translation consiste nella traduzione automatica di testo operata da modelli *encoder-decoder* o *decoder-only*. Le lingue condividono caratteristiche universali, ma presentano differenze fondamentali che il modello deve gestire, come l'ordinamento delle parole (SVO, SOV, ecc.), divergenze lessicali (sinonimi), tipologie morfologiche e densità referenziale (ad esempio, l'omissione dei pronomi in italiano rispetto all'inglese).



**Strategie di Decoding** Nella fase di generazione del testo, l'obiettivo è trovare la sequenza di token più probabile. La *greedy search* sceglie il token più probabile ad ogni passo, offrendo una soluzione localmente ottimale ma non necessariamente globale. Poiché la ricerca esaustiva è impraticabile a causa dell'enorme spazio delle soluzioni, si adottano strategie avanzate:

- **Beam Search:** Ad ogni step, per ciascuna delle  $k$  ipotesi, si calcola la probabilità dei token successivi e si mantengono solo le  $k$  sequenze con probabilità cumulativa maggiore. Lo *score* di una sequenza generata è dato dalla somma delle log-probabilità; poiché la natura additiva dello score penalizzerebbe le sequenze più lunghe, si applica un fattore di normalizzazione basato sulla lunghezza  $T$  della sequenza:

$$score(y) = \frac{1}{T} \sum_{i=1}^T -\log P(y_i | y_1, \dots, y_{i-1}, x)$$

- **Minimum Bayes Risk (MBR) Decoding:** Strategia che utilizza esplicitamente una metrica di valutazione (*util*) per misurare la bontà di una traduzione. Piuttosto che scegliere la traduzione più probabile, l'MBR seleziona quella con l'errore atteso più basso rispetto ad un set  $Y$  di traduzioni candidate:

$$\hat{y} = \underset{y \in Y}{\operatorname{argmax}} \sum_{c \in Y} util(y, c)$$

#### 11.1.2 Text Classification e Question Answering

Task tradizionalmente discriminativi, come la *Text Classification* (es. sentiment analysis, fake news detection), possono essere risolti in ottica generativa tramite *prompting* o *constraint decoding* (forzando il modello a generare solo le etichette valide).

Il **Question Answering (QA)** prevede due approcci principali:

- *Extractive*: Individuare la sottosequenza esatta che contiene la risposta all'interno di un contesto fornito.
- *Abstractive*: Generare testualmente una risposta a partire da un contesto (Open QA).

Spesso i sistemi QA si focalizzano su *factoid questions* (domande su fatti specifici). Tuttavia, demandare il task di QA a un LLM in forma puramente generativa comporta un alto rischio di *allucinazioni* (risposte non fattuali). Per mitigare questo rischio, la conoscenza parametrica del modello viene spesso affiancata a sistemi di Information Retrieval (architetture RAG).

## 11.2 Valutazione dei Modelli Generativi

La valutazione stima la bontà del testo generato, idealmente confrontandolo con un corpus di frasi *golden* o *reference*. Le due proprietà principali valutate sono l'*Adequacy/Faithfulness* (fedeltà al significato) e la *Fluency* (correttezza grammaticale).

Le tipologie di valutazione si dividono in: Umana, Automatica e LLM as a judge. La valutazione umana, seppur costosa, è utile quando manca una *golden answer* e permette di ottenere *score* numerici o *ranking* di alta qualità.

### 11.2.1 Metriche di Valutazione Automatica

Confrontano la risposta generata (*hypothesis*,  $h$ ) con una o più frasi di *reference* ( $r$ ). Possono basarsi sulla sovrapposizione dei caratteri/parole o sulla similarità degli *embedding*:

- **chrF (Character F-score)**: Misura la sovrapposizione di  $k$ -grams di caratteri. Calcola la media delle *precision* (chrP) e delle *recall* (chrR) calcolando una F1-score pesata dal parametro  $\beta$ :

$$chrF\beta = (1 + \beta^2) \frac{chrP \cdot chrR}{\beta^2 \cdot chrP + chrR}$$

- **BLEU (BiLingual Evaluation Understudy)**: Metrica orientata alla *precision* basata sulla sovrapposizione di parole (n-grams). È moltiplicata per una *Brevity Penalty* (BP) per penalizzare frasi troppo corte:

$$BLEU(H, R) = BP \cdot \exp \left( \frac{1}{N} \sum_{n=1}^N \log p_n(H, R) \right)$$

- **ROUGE**: Metriche orientate alla *recall*. *ROUGE-N* calcola precision e recall degli N-grammi comuni. *ROUGE-L* ricerca la Longest Common Subsequence (LCS) tra hypothesis e reference.
- **METEOR**: Supera i limiti della sovrapposizione esatta considerando sinonimia, *stemming* e l'ordine delle parole. Utilizza una formula basata su F-mean e una penalità.

$$Score = F_{mean} * (1 - Penalty)$$

- **BERTScore**: Sposta la comparazione nello spazio degli embedding contestuali, calcolando la Pairwise Cosine Similarity tra i token generati e quelli di reference. La *recall* del BERTScore è:

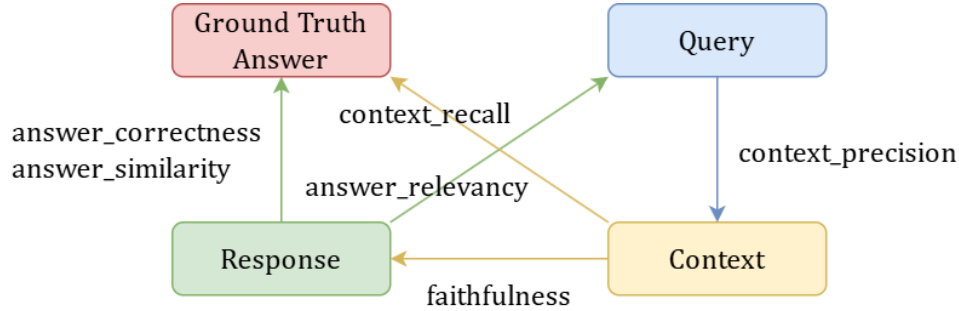
$$R_{BERT} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\tilde{x}_j \in \tilde{x}} x_i \cdot \tilde{x}_j$$

### 11.2.2 LLM as a Judge

Tecnica in cui un LLM esperto (es. GPT-4, Claude) viene interrogato per valutare l'output generato da un altro modello. Formalmente, la valutazione  $\mathcal{E}$  è espressa come:  $\mathcal{E} \leftarrow P_{LLM}(x \oplus \mathcal{C})$  dove  $x$  è l'input da valutare,  $\mathcal{C}$  è il contesto/template (che può richiedere una classificazione binaria, un confronto o una scala Likert), e  $\oplus$  è l'operatore di concatenazione.

### 11.2.3 RAGAS

RAGAS è un framework basato su "LLM as a judge" specifico per valutare sistemi RAG (Retrieval-Augmented Generation). Valuta quattro metriche principali:



1. **Context Recall:** Misura la proporzione di informazioni rilevanti (Ground Truth) recuperate con successo.

$$Context\_Recall = \frac{|GT \text{ claims attributed to the context}|}{|GT \text{ claims}|}$$

2. **Context Precision:** Proporzione di *chunk* rilevanti nel contesto recuperato, calcolata come media della *Precision@k* per i risultati top *K*.
3. **Faithfulness:** Accuratezza fattuale; verifica se le affermazioni della risposta sono supportate dal contesto fornito.

$$Faithfulness = \frac{\# \text{ valid statements}}{\# \text{ statements}}$$

4. **Answer Relevancy:** Misura la pertinenza della risposta calcolando la *cosine similarity* tra la domanda originale ( $E_o$ ) e un set di  $N$  domande generate a partire dalla risposta ( $E_{g_i}$ ).

$$Answer\_Relevancy = \frac{1}{N} \sum_{i=1}^N \frac{E_{g_i} \cdot E_o}{||E_{g_i}|| ||E_o||}$$



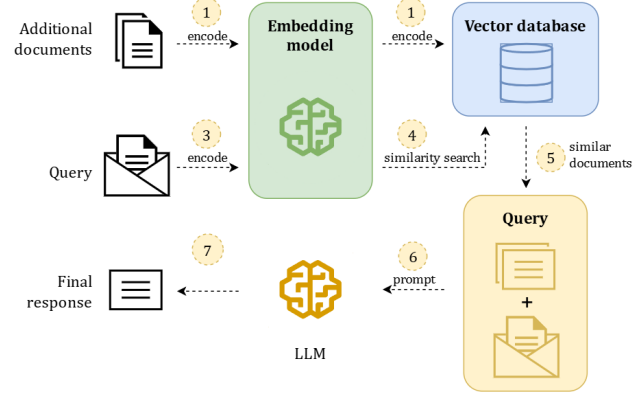
## 12 Retrieval-Augmented Generation

La *Retrieval-Augmented Generation* (RAG) è una tecnica avanzata che combina il recupero di informazioni (*retrieval*) e la generazione di testo (*generation*) per migliorare le capacità di un Large Language Model (LLM) senza la necessità di effettuare il *fine-tuning* del modello stesso.

Rispetto a un Base LLM (soggetto ad allucinazioni e privo di informazioni aggiornate) o a un FT LLM (che richiede costi elevati di addestramento), un sistema RAG offre risposte di alta qualità, accesso a informazioni aggiornate e un maggior controllo sui dati generati, il tutto con costi contenuti (*on-budget*).

Dal punto di vista concettuale, l'architettura RAG si basa su due tipi di memoria:

- **Memoria non-parametrica:** Gestita dal *retriever*, che è statico e non possiede pesi apprendibili.
- **Memoria parametrica:** Gestita dal *generator* (il LLM), la cui conoscenza dipende dai pesi appresi in fase di addestramento.



### 12.1 Componenti dell'Architettura RAG

#### 12.1.1 Corpus ed Embeddings

I dati di dominio (come documenti testuali, JSON o dati privati) costituiscono la *Knowledge Base*. Questi documenti vengono suddivisi in piccole sezioni chiamate *chunk*. Successivamente, un modello linguistico dedicato (*Embedding LLM*) codifica i chunk trasformandoli in rappresentazioni vettoriali dense (*embedding*).

#### 12.1.2 Retriever e Vector Store

L'obiettivo del *retriever* è estrarre i documenti più rilevanti per una data *query* utente, al fine di arricchire il contesto del generatore. I chunk vettorializzati vengono memorizzati in un *vector store* (come Chroma, Faiss, Pinecone o Neo4j), un database indicizzato che permette memorizzazione e accesso efficienti.

Il recupero avviene tramite una *similarity function* che ordina i migliori  $k$  chunk estratti. Oltre alla classica *cosine similarity*, si utilizza spesso il criterio della *Maximum Marginal Relevance (MMR)* per bilanciare la rilevanza rispetto alla query e la diversità dei documenti estratti. La formula del MMR è:

$$MMR = \arg \max_{D_i \in R \setminus S} \left[ \lambda \cdot Sim_1(D_i, Q) - (1 - \lambda) \cdot \max_{D_j \in S} Sim_2(D_i, D_j) \right]$$

dove:

- $Q$  è la query dell'utente.
- $R$  è la lista dei chunk candidati recuperati dal retriever.
- $S$  è l'insieme dei documenti già selezionati per il contesto.
- $\lambda \in [0, 1]$  regola il compromesso:  $\lambda = 1$  massimizza la rilevanza rispetto alla query, mentre  $\lambda = 0$  favorisce la massima diversità.

#### 12.1.3 Generator

Il generatore è tipicamente un LLM *decoder-only*. Riceve in input un *prompt* strutturato contenente l'istruzione di base, il contesto (i documenti recuperati) e la domanda dell'utente. Basandosi esclusivamente su tali informazioni, il modello genera la risposta finale.

## 12.2 Valutazione dei Sistemi RAG

La valutazione di un sistema RAG si divide tra le prestazioni del recupero e quelle della generazione:

- **Retriever:** Si valuta la *Precision*, l'*Accuracy* e la pertinenza del contesto (*Context relevancy*), spesso tramite tecniche di *LLM as a judge* o valutazione umana.
- **Generator:** Si valuta la qualità testuale con metriche tradizionali (BLEU, ROUGE, BERT-score, Perplexity) e l'affidabilità (*Faithfulness*, *correctness*) per assicurarsi che il testo generato non contenga allucinazioni rispetto al contesto fornito.

## 12.3 Integrazioni Avanzate: Grafi e Agentic RAG

### 12.3.1 RAG e Knowledge Graphs

Per basi di dati complesse, i dati possono essere organizzati in strutture a grafo (*Knowledge Graphs*), dove le informazioni sono espresse in triplette logiche *(soggetto, relazione, oggetto)*. I *graph database* mantengono l'architettura e la semantica dei dati attraverso ontologie e linguaggi di query specifici (es. Cypher o SparQL). Rispetto al vector store tradizionale, i grafi evidenziano esplicitamente le relazioni tra entità, permettendo al sistema RAG di estrarre e usare come contesto dati maggiormente strutturati e "collegare i puntini" con più efficacia.

### 12.3.2 Agentic RAG

Il paradigma dell'*Agentic RAG* integra il RAG all'interno di un'architettura ad agenti, in cui uno o più LLM (Single Agent o Multi-agent) cooperano per svolgere dinamicamente il task di recupero. In queste architetture, il modello utilizza tool (es. Vector Search, Web Search, API), memoria (Short-term e Long-term) e meccanismi di pianificazione, riflessione e scomposizione dei compiti per elaborare una o più query ottimali e generare la risposta. Le librerie più comuni per sviluppare queste pipeline complesse sono **LangChain** e **LlamaIndex**.